



Adji-SP

Membangun Dashboard Profiling CSV dan OTA ESP32

Panduan bersahabat dari klik pertama
sampai pembaruan pertama

EDISI BAHASA INDONESIA

CSV_PROFILING



Membangun Dashboard Profiling CSV dan OTA ESP32

Edisi Bahasa Indonesia, September 2026.

Buku ini mendokumentasikan proyek lokal sebagaimana diterapkan di repositori ini. PDF *beautybook* yang diberikan menjadi inspirasi untuk tata visualnya. Seluruh penjelasan dan contoh ditulis khusus untuk CSV_PROFILING.

Perintah yang menyentuh perangkat keras diberi tanda yang jelas. Periksa ukuran flash ESP32-S3, port serial, tabel partisi, dan pengaturan jaringan lokal sebelum menjalankannya.



KATA PENGANTAR

Saat folder proyek dibuka untuk pertama kali, rasanya seperti ada tiga pelajaran yang ditumpuk di atas satu meja: analisis data, pengembangan web, dan Rust untuk perangkat embedded. Kita tidak perlu mengangkat semuanya sekaligus. Ambil satu bagian kecil, pahami tugasnya, lalu sambungkan dengan bagian berikutnya.

Di belakang dashboard terdapat dua ruang kerja. Flask dan Pandas mengurus berkas CSV. Axum mengurus build firmware dan perangkat. Lebih jauh lagi, sebuah ESP32-S3 terhubung melalui Wi-Fi dan bertanya kepada server apakah ada aplikasi baru untuknya. Setelah gambaran tiga ruang ini terasa akrab, susunan folder tidak lagi tampak seperti dinding penuh nama asing.

Gunakan buku ini sambil membuka proyek di sebelahnya. Baca beberapa halaman, sentuh berkas yang sedang dibicarakan, lalu kerjakan pemeriksaan kecil di akhir bab. Biarkan terminal tetap terlihat. Pesan gagal pada sebuah perintah sering kali menunjukkan pintu mana yang masih terkunci.

Proyek CSV_PROFILING

2026



CARA MENGGUNAKAN BUKU INI

Anggap halaman-halaman ini sebagai buku catatan bengkel yang penuh coretan penunjuk. Kode tidak perlu dihafalkan. Buka repositori, ikuti jalur berhuruf monospace, lalu bandingkan contoh pendek dengan berkas aslinya. Perintah Windows memakai PowerShell. Perintah Linux dan macOS memakai shell POSIX.

Di sepanjang pelajaran, Anda akan menjumpai tiga jenis catatan:

Catatan dari meja tutor

Catatan tutor memperlambat langkah untuk menjawab pertanyaan yang biasanya muncul saat dua orang duduk di meja yang sama.

Berhenti sebentar sebelum lanjut

Catatan berhenti muncul sebelum sebuah perintah dapat menyentuh perangkat keras, menghapus data, atau melonggarkan batas keamanan.

Coba dahulu sebelum lanjut

Latihan kecil membiarkan layar Anda sendiri membuktikan penjelasan yang baru dibaca.

Anda dapat mulai walau ESP32 belum ada di atas meja. Dashboard, API, basis data, pemeriksaan ZIP, dan pesan kesalahan berjalan di komputer. Ketika pelajaran sampai pada proses flash fisik, buku akan mengatakannya dengan terang.

DAFTAR ISI

I	Dasar-Dasar	1
1	Berkenalan dengan Proyek Sebelum Menyentuh Kode	3
1.1	Satu dashboard, dua pekerja	3
1.2	Ikuti perjalanan berkas CSV dengan jari	4
1.3	Sekarang ikuti perjalanan ZIP firmware	4
1.4	Gunakan folder sebagai papan penunjuk	4
1.5	Program main yang mana yang benar-benar utama?	5
1.6	Mengapa flash pertama memakai USB	5
1.7	Siapa yang memegang modem Wi-Fi?	6
2	Mari Menyalakan Proyeknya	7
2.1	Mulai dari komputer	7
2.2	Siapkan Python dengan tenang	7
2.3	Periksa alat Rust yang sudah tersedia	7
2.4	Buat berkas .env milik Anda	8
2.5	Tekan tombol mulai milik proyek	8
2.6	Ajukan satu pertanyaan kecil kepada tiap layanan	9
2.7	Jika halaman menolak terbuka	9
II	Dashboard dan CSV Profiler	10
3	Berjalan Menyusuri Dashboard	12
3.1	Mulai dari apa yang terlihat oleh mata	12
3.2	Kenali berkas frontend satu per satu	12
3.3	Sebuah tombol menjalankan rantai peristiwa pendek	13
3.4	Mengapa ada dua alamat API	13
3.5	Perhatikan suasana halaman build OTA berubah	13
3.6	Memahami kartu perangkat	14

3.7	Buat satu perubahan visual yang aman	14
4	Mengikuti Satu Berkas CSV	15
4.1	Buat berkas kecil yang dapat dipahami dengan mata	15
4.2	Route upload menerima amplopnya	15
4.3	Progress datang seperti catatan di bawah pintu	16
4.4	Tempat laporan beristirahat	16
4.5	Bersihkan salinan, lalu lihat lagi	17
4.6	Gunakan playground sebagai kertas coretan	17
4.7	Menambah fitur CSV tanpa mengganggu OTA	17
III	Layanan OTA Rust	19
5	Berkenalan dengan Server OTA Rust	21
5.1	Baca main.rs seperti rutinitas membuka bengkel	21
5.2	Konfigurasi adalah halaman tata tertib rumah	21
5.3	Route adalah label di meja penerima	22
5.4	Mengapa browser diperbolehkan masuk	22
5.5	Pesan kesalahan perlu berbicara dengan kalimat lengkap	23
5.6	SQLite adalah buku catatan server	23
6	Apa yang Sebenarnya Terjadi pada ZIP	25
6.1	Mulai dari formulir upload	25
6.2	Server memeriksa koper sebelum membukanya	25
6.3	Proyek memilih satu dari dua jalur	25
6.4	Bagaimana dua berkas bernama main.rs hidup bersama	26
6.5	Builder memeriksa alatnya	26
6.6	Cargo mengubah source menjadi ELF	27
6.7	esptool menyiapkan image OTA	27
6.8	Sidik jari menutup paket	27
6.9	Baca status build sebagai cerita pendek	27
IV	Firmware di ESP32-S3	29
7	Program yang Tinggal di Dalam ESP32	31

7.1	Berikan fondasi yang dapat diandalkan kepada board	31
7.2	Perjalanan pertama melewati USB	31
7.3	Baca fn main yang sebenarnya seperti sebuah cerita	32
7.4	Buka crate yang lebih kecil	32
7.5	AppContext adalah tas serah terima	33
7.6	Flash baru tidak menumpuk kode lama	33
8	Menulis Firmware Upload Pertama	34
8.1	Salin contoh kecil sebelum memulai dari halaman kosong	34
8.2	Program Anda tetap terasa seperti main.rs	34
8.3	Nilai antara setup dan main adalah kotak perkakas	35
8.4	Ambil peripheral saat setup	35
8.5	Tambahkan library Cargo biasa	36
8.6	Berikan versi yang berguna kepada rilis	36
8.7	Zip isinya, jangan rak luarnya	37
9	Melihat Perjalanan Sebuah Update OTA	38
9.1	Dashboard meninggalkan pesan untuk perangkat	38
9.2	Perangkat memperkenalkan diri	38
9.3	Pemeriksaan update mengembalikan manifest kecil	39
9.4	Binary berpindah satu cangkir setiap kali	39
9.5	Perangkat menceritakan jalannya update	39
9.6	Lihat isi tabel partisi dua laci	40
9.7	Rollback memberi masa percobaan kepada image baru	40
V	Latihan dan Pengembangan Lanjutan	42
10	Menyatukan Seluruh Alur	44
10.1	Kosongkan sedikit ruang di meja	44
10.2	Tahap satu: bangunkan kedua layanan PC	44
10.3	Tahap dua: periksa kesehatan jalur CSV lama	44
10.4	Tahap tiga: bungkus aplikasi basic	45
10.5	Tahap empat: periksa hasil build yang berhasil	45
10.6	Tahap lima: berikan program dasar pertama kepada board	45
10.7	Tahap enam: tinggalkan pesan update	46

10.8 Tahap tujuh: ubah sesuatu yang mudah dikenali	46
10.9 Tuliskan bukti selagi masih segar	46
11 Ketika Sesuatu Tidak Berjalan	47
11.1 Duduk sebentar di langkah terakhir yang berhasil	47
11.2 Halaman dashboard tidak dapat dibuka	47
11.3 CSV bekerja dan OTA mengatakan offline	47
11.4 ZIP ditolak di depan pintu	48
11.5 Build berhenti sebelum kode Anda muncul	48
11.6 Rust memberi satu halaman error compiler	48
11.7 Cargo berhasil dan pembuatan image gagal	48
11.8 PC melihat firmware, board tidak melihatnya	49
11.9 Kartu perangkat tidak pernah muncul	49
11.10Kartu tetap berada pada Pending	49
11.11Ukuran atau SHA-256 tidak cocok	49
11.12Binary sudah terlalu besar untuk lacinya	49
11.13Delete menjawab 409 Conflict	49
11.14Di mana pesan terminal ESP32?	50
12 Arah Pengembangan Berikutnya	51
12.1 Pilih satu sambungan dan biarkan bagian lain tenang	51
12.2 Sambut chip ESP32 lain	51
12.3 Ubah setup Wi-Fi menjadi alur pertama yang ramah	51
12.4 Bawa terminal ESP32 ke dashboard	52
12.4.1 Jalan pertama: serial bridge pada PC	52
12.4.2 Jalan kedua: log dikirim melalui Wi-Fi	52
12.5 Pindahkan build yang tidak dipercaya ke ruang lebih aman	52
12.6 Berikan tanda tangan sungguhan kepada firmware	53
12.7 Biarkan API tumbuh tanpa mengejutkan proyek lama	53
12.8 Uji dari lapisan murah menuju board	53
A Referensi HTTP API	54
A.1 Konvensi	54
A.2 Indeks endpoint OTA	55
A.3 Status	55
A.4 Project dan build	55

A.5	Firmware	56
A.6	Perangkat dan deployment	57
A.7	Indeks endpoint CSV	58
B	Konfigurasi, Perintah, dan Glosarium	60
B.1	Environment variable	60
B.2	Lembar perintah	61
B.2.1	Menyalakan layanan	61
B.2.2	Menjalankan pemeriksaan server OTA	61
B.2.3	Memeriksa alat embedded	61
B.2.4	Membangun runtime stabil tanpa flashing	61
B.2.5	Membangun dan melakukan flash runtime stabil	61
B.3	Glosarium	62
B.4	Batas implementasi saat ini	62
	Catatan Penutup	63

BAGIAN I

DASAR-DASAR



Kita mulai dari ambang pintu. Anda akan melihat bentuk utuh proyek dengan bahasa sederhana, lalu menyalakannya pelan-pelan sambil membaca tanda-tanda kecil yang menunjukkan bahwa semuanya sehat.

1

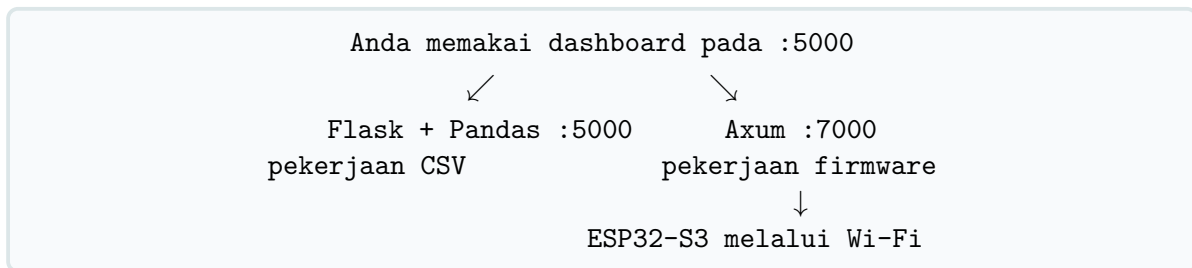
BERKENALAN DENGAN PROYEK SEBELUM MENYENTUH KODE

Bag. 1.1 Satu dashboard, dua pekerja

Buka dashboard di browser. Seluruh halaman memakai sidebar, warna, dan tombol yang sama, sehingga semuanya tampak seperti satu aplikasi. Di balik layar itu ada dua pekerja yang mempunyai meja masing-masing.

Pekerja Python mendengarkan pada port 5000. Berikan sebuah berkas CSV, lalu ia akan membaca baris-barisnya, menghitung ringkasan, membuat laporan, dan menerapkan pilihan pembersihan data. Pekerja Rust mendengarkan pada port 7000. Berikan sebuah proyek firmware, lalu ia akan menjaga proses upload, menjalankan alat build, menyimpan binary yang selesai, dan mengingat perangkat mana yang perlu menerimanya.

Bayangkan sebuah bengkel kecil dengan satu meja penerima tamu dan dua meja kerja. Browser adalah meja penerima tamu. Meja pertama menangani data. Meja kedua menangani firmware.



Pemisahan meja ini melindungi CSV profiler yang sudah bekerja. Penambahan fitur firmware tidak memaksa kita membangun ulang layanan data dengan Rust. Browser hanya perlu mengetahui alamat mana yang harus dipanggil.

Sebuah istilah yang akan sering muncul

Pengembang menyebut serah terima antara dua program sebagai *service boundary*, atau batas layanan. Di proyek ini, batas tersebut hanyalah sebuah permintaan HTTP. Istilahnya terdengar resmi, tetapi gambarnya sederhana: satu program meminta program lain mengerjakan tugas yang namanya jelas.

Bag. 1.2 Ikuti perjalanan berkas CSV dengan jari

Misalkan Anda memilih `readings.csv`, lalu menekan Run Profiling. Browser mengirim berkas itu ke Flask. Flask menyimpan salinan kerja dan memulai pekerjaan profiling yang lebih lambat di latar belakang. Ketika Pandas membaca data, browser menerima kabar kemajuan. Setelah laporan selesai, Flask menyajikan halaman HTML hasilnya.

Setiap tindakan CSV berikutnya kembali kepada pekerja yang sama. Previous Runs meminta daftar sesi tersimpan dari Flask. Cleaning meminta Flask mengubah salinan kerja. Playground meminta Flask menjalankan satu operasi Pandas kecil.

Jalurnya dapat diingat dalam satu baris:

`browser → Flask → Pandas → laporan`

Bag. 1.3 Sekarang ikuti perjalanan ZIP firmware

Perjalanan OTA dimulai dengan ZIP karena sebuah program Rust terdiri dari lebih dari satu berkas sumber. Di dalamnya dapat ada manifest Cargo, modul tambahan, dan dependensi library. Browser mengirim ZIP tersebut bersama pilihan board dan nomor versi kepada Axum.

Axum memeriksa arsip sebelum membukanya. Setelah aman, Axum meminta Cargo mengompilasi proyek untuk ESP32-S3. Cargo menghasilkan executable ELF. `espflash` mengubah ELF itu menjadi image aplikasi ESP-IDF. Server menghitung sidik jari SHA-256 dan menyimpan binary di dalam `firmware-builds`.

Nanti, perangkat bertanya apakah ada pembaruan yang menunggunya. Jika pembaruan sudah ditugaskan, ESP32 mengunduh berkas `.bin`. ESP32 tidak pernah menerima ZIP ataupun berkas `.rs` yang berdiri sendiri.

`ZIP → Cargo → ELF → aplikasi .bin → ESP32`

Bag. 1.4 Gunakan folder sebagai papan penunjuk

Simpan tabel ini di dekat Anda selama beberapa sesi pertama. Untuk awal, cukup pahami kalimat pertama pada setiap baris.

Folder atau berkas	Isi yang tinggal di sana
<code>web/</code>	Halaman yang terlihat: HTML, CSS, dan JavaScript browser.
<code>python/profiler_server.py</code> <code>ota-server/</code>	Route Flask dan seluruh pekerjaan CSV yang sudah ada. Server Axum, kode basis data, pemeriksaan ZIP, runner build, dan API perangkat.
<code>firmware-projects/</code>	Folder kerja privat yang dibuat saat Axum mengekstrak upload.
<code>firmware-builds/</code>	Binary aplikasi OTA yang sudah selesai.

Folder atau berkas	Isi yang tinggal di sana
<code>data/ota.db</code>	Buku catatan SQLite tempat Axum mengingat proyek, build, firmware, perangkat, dan deployment.
<code>examples/esp32-rust-ota-client/</code>	Program dasar yang stabil, dipasang pada board dan dipakai kembali dalam managed build.
<code>examples/uploaded-firmware-basic/</code>	Aplikasi paling kecil yang dapat dipelajari dan diunggah.
<code>examples/uploaded-firmware-with-libs/</code>	Aplikasi kedua yang memperlihatkan library Cargo tambahan.
<code>docs/OTA_GUIDE.md</code>	Catatan teknis yang lebih singkat tentang partisi, roll-back, dan jaringan.

Ketiga folder penyimpanan akan terisi sendiri saat sistem berjalan. Kode yang Anda tulis dengan tangan berada di `web`, `python`, `ota-server`, atau sebuah proyek aplikasi di bawah `examples`.

Bag. 1.5 Program main yang mana yang benar-benar utama?

Ada tiga program yang berjalan di tiga tempat. Browser menjalankan JavaScript. PC menjalankan Flask dan Axum. ESP32 menjalankan satu image firmware hasil kompilasi.

Di dalam image firmware itu, `examples/esp32-rust-ota-client/src/main.rs` memiliki entripoint asli `fnmain()`. Aplikasi yang Anda unggah juga menyimpan berkas bernama `src/main.rs`, tetapi berkas tersebut mengeksport dua fungsi yang dapat dipanggil, yaitu `setup` dan `main`. Saat build, Rust menautkan kedua fungsi itu ke runtime yang stabil.

Bayangkan runtime sebagai sebuah rumah yang kabel listrik dan saluran airnya sudah terpasang. Aplikasi Anda menentukan kegiatan di dalam ruangnya. Hasil build tetap berupa satu rumah lengkap. Tidak ada program kedua yang ditumpuk di atas program pertama.

Bag. 1.6 Mengapa flash pertama memakai USB

ESP32 baru belum mengetahui nama Wi-Fi Anda. Ia belum memiliki alamat server OTA dan belum mempunyai tabel partisi OTA dua slot. USB memberinya fondasi pertama: bootloader, partisi, runtime yang memahami Wi-Fi, dan sebuah aplikasi bawaan kecil.

Sesudah flash pertama berhasil, board dapat memperkenalkan diri kepada server dan meminta rilis berikutnya melalui Wi-Fi.

Jawaban singkatnya

Ya, flash runtime stabil ke ESP32-S3 terlebih dahulu. Biasanya langkah ini dilakukan sekali ketika board disiapkan. Setelah perangkat mampu bergabung ke jaringan dan menjangkau port 7000 pada PC, OTA dapat mengambil alih pembaruan berikutnya.

Bag. 1.7 Siapa yang memegang modem Wi-Fi?

ESP32 hanya memiliki satu sumber daya modem. Runtime mengambilnya satu kali, lalu menyerahkannya kepada `wifi-manager`. Manager tersebut menjaga koneksi untuk heartbeat dan pemeriksaan OTA. Aplikasi Anda menerima keadaan jaringan melalui `AppContext`, bersama pin dan pengendali hardware lain yang boleh dipakai.

Susunan ini menyediakan tempat untuk layar pengaturan Wi-Fi yang lebih ramah di masa depan. Kredensial tersimpan, aturan reconnect, atau captive portal dapat tinggal di dalam manager. Aplikasi sensor tetap dapat memusatkan perhatian pada sensor.

Berhenti sebentar sebelum lanjut

Jangan memanggil `Peripherals::take()` dari aplikasi yang diunggah. Runtime sudah mengambil kumpulan global tersebut. Gunakan `context.peripherals`, lalu ambil hanya pin atau controller yang benar-benar diperlukan aplikasi.

Coba dahulu sebelum lanjut

Buka repositori dan cari empat tempat ini: `web/index.html`, `python/profiler_server.py`, `ota-server/src/main.rs`, dan `examples/esp32-rust-ota-client/src/main.rs`. Ucapkan mesin mana yang menjalankan masing-masing berkas. Kalimat kecil itu akan mencegah banyak kebingungan di bab berikutnya.

2

MARI MENYALAKAN PROYEKNYA

Bag. 2.1 Mulai dari komputer

Biarkan ESP32 belum terhubung untuk sementara. Sasaran pertama lebih kecil: buka dashboard dan pastikan kedua layanan di PC menjawab. Sesudah ini berhasil, masalah pada langkah berikutnya mempunyai lebih sedikit tempat untuk bersembunyi.

Sisi CSV membutuhkan Python 3.10 atau yang lebih baru. Sisi OTA membutuhkan Rust. Toolchain embedded dapat menunggu sampai Anda merasa nyaman menyalakan dashboard.

Bag. 2.2 Siapkan Python dengan tenang

Virtual environment memberi proyek ini kotak paket Python miliknya sendiri. Proyek Python lain di komputer tidak akan tercampur dengannya.

Buka PowerShell di folder repositori, lalu ketik baris pertama:

```
1 py -3 -m venv python\.venv
```

Setelah perintah selesai, lihat isi folder `python`. Seharusnya ada folder baru bernama `.venv`. Sekarang pasang paket yang tercantum di `python/requirements.txt`:

```
1 python\.venv\Scripts\python -m pip install -r python/requirements.txt
```

Di Linux atau macOS, dua langkah yang sama ditulis seperti ini:

```
1 python3 -m venv python/.venv
2 python/.venv/bin/pip install -r python/requirements.txt
```

Pemasangan dapat memakan waktu karena library profiling data membawa beberapa paket pendamping. Tunggu sampai prompt perintah kembali. Teks merah di tengah proses kadang hanya peringatan. Bacalah baris paling akhir untuk mengetahui apakah pemasangan berhasil atau berhenti karena kesalahan.

Bag. 2.3 Periksa alat Rust yang sudah tersedia

Jalankan satu perintah setiap kali:

```
1 cargo --version
2 rustc --version
```

Jika keduanya mencetak nomor versi, PC sudah dapat mulai membangun server Axum. Firmware memerlukan dua pemeriksaan tambahan nanti:

```
1 rustc +esp --version
2 espflash --version
```

Anda belum perlu mengejar seluruh alat embedded yang hilang. Dashboard tetap dapat menyala, menyimpan proyek, memeriksa ZIP, dan menampilkan pesan build gagal yang jujur.

Detail Windows yang sering mengejutkan

Toolchain ESP Rust mungkin sudah terpasang sementara Windows masih belum mempunyai `link.exe`. Berkas itu berasal dari Visual Studio Build Tools dengan workload Desktop development with C++. Jika log menyebut `link.exe`, kode Rust belum tentu salah. Compiler sedang meminta alat native yang tidak ditemukan di Windows.

Bag. 2.4 Buat berkas `.env` milik Anda

Salin `.env.example`, lalu beri nama salinannya `.env`. Inilah halaman pengaturan lokal Anda. Berkas tersebut dapat menyimpan kata sandi Wi-Fi, sehingga Git harus mengabaikannya.

Untuk penyalaan pertama, perhatikan baris-baris ini:

```
OTA_BIND_ADDRESS=0.0.0.0
OTA_PORT=7000
OTA_PUBLIC_BASE_URL=http://192.168.1.5:7000

WIFI_SSID=your-wifi-name
WIFI_PASS=your-wifi-password
DEVICE_NAME=ESP32-S3 Managed Device
```

Ganti 192.168.1.5 dengan alamat LAN PC Anda. Di Windows, jalankan `ipconfig`, lalu lihat adapter Wi-Fi atau Ethernet yang sedang dipakai. Alamat yang dicari biasanya diawali 192.168 atau 10.

Mengapa localhost membuat perangkat tersesat

Di PC, `localhost` berarti PC tersebut. Di ESP32, kata yang sama berarti ESP32 itu sendiri. Masukkan alamat LAN PC ke `OTA_PUBLIC_BASE_URL` dan `OTA_SERVER_URL` milik runtime.

Nilai lain di `.env.example` sudah menunjuk ke folder lokal yang wajar. Biarkan dahulu selama percobaan pertama.

Bag. 2.5 Tekan tombol mulai milik proyek

Pengguna Windows dapat menjalankan:

```
1 .\run-dev.bat
```

Dua jendela terminal akan terbuka. Satu bertuliskan CSV Profiler dan memakai port 5000. Jendela lain mengompilasi lalu menyalakan server OTA Rust pada port 7000. Build Rust pertama mungkin diam

cukup lama karena Cargo sedang menyiapkan dependensi.

Pengguna Linux dan macOS dapat menjalankan:

```
1 chmod +x run-dev.sh
2 ./run-dev.sh
```

Ketika kedua pekerja siap, buka `http://localhost:5000`. Sidebar dan halaman upload CSV seharusnya terlihat.

Bag. 2.6 Ajukan satu pertanyaan kecil kepada tiap layanan

Dashboard membuktikan bahwa Flask menjawab. Berikan tes kecil yang serupa kepada Axum dengan membuka:

```
http://localhost:7000/api/ota/status
```

Browser akan menampilkan JSON. Tidak perlu memahami seluruh field dahulu. Cari `"status": "online"`, lalu pastikan `public_base_url` berisi alamat LAN Anda.

```
{
  "service": "CSV_PROFILING OTA Server",
  "status": "online",
  "public_base_url": "http://192.168.1.5:7000",
  "supported_targets": ["esp32s3"]
}
```

Bag. 2.7 Jika halaman menolak terbuka

Jangan langsung menyalakan ulang semuanya. Periksa satu pintu setiap kali.

Pertama, kunjungi `http://127.0.0.1:5000`. Jika alamat itu juga gagal, baca terminal Flask. Paket Python yang hilang biasanya menyebut namanya sendiri di sana.

Jika dashboard terbuka tetapi lampu OTA menunjukkan offline, kunjungi URL status pada port 7000. Baca terminal Rust jika alamat itu gagal. Cargo mungkin masih mengompilasi, program lain mungkin memakai port tersebut, atau konfigurasi mungkin menghentikan startup.

Jika kedua alamat bekerja di PC namun ESP32 kelak tidak dapat terhubung, arahkan perhatian ke alamat LAN, firewall, dan jaringan Wi-Fi. Itu pintu yang berbeda.

Buka dashboard melalui HTTP. Membuka `web/index.html` dengan klik ganda menghasilkan halaman `file:///`. Origin browser-nya berbeda dan dapat membuat kode API yang sehat tampak rusak.

Coba dahulu sebelum lanjut

Biarkan dua tab terbuka. Tab pertama berisi `http://localhost:5000`. Tab kedua berisi JSON status Axum. Cocokkan keduanya dengan dua jendela terminal. Sekarang Anda mempunyai garis awal yang bersih, bahkan tanpa ESP32.

BAGIAN II

DASHBOARD DAN CSV PROFILER



Satu berkas CSV akan menjadi pemandu. Kita akan mengikutinya dari tombol upload menuju Python, melihat laporan kembali ke browser, lalu mengintip cara setiap layar dashboard bekerja.

BERJALAN MENYUSURI DASHBOARD

Bag. 3.1 Mulai dari apa yang terlihat oleh mata

Biarkan dashboard terbuka di sebelah folder proyek. Jalur gelap di kiri adalah sidebar. Bilah tipis di atas menunjukkan halaman yang sedang aktif. Area terang yang luas menjadi tempat tugas saat ini muncul.

Klik CSV Profiler, Previous Runs, OTA Firmware, Devices, dan Build History. Browser tidak memuat lima berkas HTML terpisah. Semua bagian sudah ada di halaman yang sama, lalu JavaScript menampilkan satu bagian pada satu waktu.

Sekarang buka `web/index.html`. Cari `ota-section`. Di sana terdapat layar OTA Firmware. Sedikit lebih bawah ada `devices-section` dan `builds-section`. HTML adalah denah lantai dashboard.

Kebiasaan kecil yang sangat membantu

Saat sebuah tombol bertingkah aneh, cari id-nya di HTML terlebih dahulu. Setelah itu, cari ID yang sama di JavaScript. Cara ini biasanya lebih cepat daripada membaca seluruh script dari baris pertama.

Bag. 3.2 Kenali berkas frontend satu per satu

Hanya empat berkas yang perlu diperhatikan:

web/index.html Benda yang ada di halaman: tautan, judul, formulir, tabel, dialog, dan keadaan kosong.

web/style.css Penampilannya: jarak, warna, border, kartu, tombol, badge, progress bar, dan terminal build yang gelap.

web/app.js Perilaku halaman CSV.

web/ota.js Perilaku OTA Firmware, Devices, dan Build History.

HTML memuat kedua script di dekat tag penutupnya:

```
<script src="app.js"></script>
<script src="ota.js"></script>
```

Urutan itu membiarkan script CSV lama tetap pada tempatnya. Perilaku OTA hadir di sebelahnya melalui berkas terpisah.

Bag. 3.3 Sebuah tombol menjalankan rantai peristiwa pendek

Lihat tombol Upload & Build di `index.html`. ID-nya adalah `ota-build-btn`. Cari tulisan itu di `ota.js`. Anda akan menemukan event handler yang membaca berkas, target, dan versi yang dipilih.

Rantainya seperti ini:

klik → baca formulir → panggil Axum → perbarui halaman

Pola yang sama muncul di seluruh dashboard. Tautan navigasi mengganti bagian yang terlihat. Tombol refresh memanggil endpoint daftar, lalu menyusun ulang tabel. Tombol deploy membuka dialog, kemudian mengirim ID firmware yang dipilih.

Anda tidak perlu memahami semua baris untuk mengikuti alurnya. Temukan benda HTML, temukan listener-nya, lalu ikuti fungsi yang dipanggil.

Bag. 3.4 Mengapa ada dua alamat API

Di bagian atas script terdapat:

```
1 const API_BASE = "http://localhost:5000";
2 const OTA_API_BASE = "http://localhost:7000";
```

Dua baris kecil ini adalah papan arah. Permintaan CSV berjalan menuju Flask. Permintaan firmware dan perangkat berjalan menuju Axum. Jangan menyatukan kedua konstanta hanya karena layanannya hidup di PC yang sama.

Script OTA membungkus pekerjaan permintaan yang berulang di dalam `otaRequest`. Dengan bahasa sehari-hari, helper itu berkata: kirim permintaan, periksa apakah berhasil, baca pesan server yang berguna jika gagal, lalu uraikan JSON ketika respons mempunyai isi.

```
1 async function getDevices() {
2   return otaRequest("/api/ota/devices");
3 }
4
5 async function deployFirmware(deviceId, firmwareId) {
6   return otaRequest(`/api/ota/devices/${deviceId}/deploy`, {
7     method: "POST",
8     headers: { "Content-Type": "application/json" },
9     body: JSON.stringify({ firmware_id: firmwareId })
10  });
11 }
```

Fungsi pertama mengajukan pertanyaan. Fungsi kedua membawa perintah. Keduanya melewati pengantar pesan yang sama.

Bag. 3.5 Perhatikan suasana halaman build OTA berubah

Pilih ZIP, lalu tekan Upload & Build. Bahkan sebelum compiler mulai, halaman bergerak melewati beberapa keadaan kecil:

Idle Belum ada pekerjaan yang dikirim.

Uploading Browser sedang mengirim arsip.

Queued Axum sudah membuat catatan build.

Building Alat-alat di PC sedang bekerja.

Success Binary sungguhan beserta sidik jarinya sudah tersimpan.

Failed Sesuatu menghentikan build, dan log menunjukkan tempatnya.

Frontend meminta catatan build setiap beberapa detik. Cara ini disebut polling. Bayangkan seseorang mengintip melalui jendela bengkel dari waktu ke waktu, alih-alih menahan sambungan telepon selama build berlangsung.

Kesalahan compiler tetap terlihat di panel terminal. Toast mungkin hanya berkata build gagal, sedangkan area log gelap menyimpan rincian yang dapat dipakai. Gulir ke atas dan cari kesalahan jelas yang pertama.

Bag. 3.6 Memahami kartu perangkat

Halaman Devices menampilkan apa yang diingat Axum. Halaman itu tidak memindai ruangan untuk mencari board Wi-Fi. ESP32 sungguhan menambahkan dirinya sendiri melalui registrasi dan heartbeat.

Setiap kartu memperlihatkan versi terakhir yang dilaporkan, versi tujuan, alamat jaringan, waktu terakhir terlihat, dan kemajuan OTA. Jika listrik board mati, kartu lamanya mungkin masih ada. Lihat timestamp untuk menilai seberapa segar informasinya.

Bag. 3.7 Buat satu perubahan visual yang aman

Buka `web/index.html`, lalu cari subtitle di bawah OTA Firmware. Ubah beberapa kata, simpan berkas, kemudian refresh dashboard. Jika tulisan lama masih ada, lakukan hard refresh agar browser berhenti memakai salinan cache.

Setelah melihat hasilnya, kembalikan kalimat latihan tadi. Anda baru saja mengikuti loop frontend paling sederhana: ubah HTML, biarkan Flask menyajikannya, lalu biarkan browser menggambarinya.

Tentang kata terminal

Terminal gelap di halaman OTA menampilkan keluaran build dari PC. Saat ini panel itu belum menampilkan pesan yang dicetak ESP32. Gunakan `esptool` untuk melihat keluaran board. Bab 12 menjelaskan dua cara yang masuk akal untuk membawa log tersebut ke dashboard di masa depan.

Coba dahulu sebelum lanjut

Pilih tautan navigasi Devices. Cari ID-nya di `index.html`, click listener di JavaScript, fungsi yang memuat perangkat, dan route Axum yang dipanggil. Tulis empat nama itu di kertas sebagai jejak kecil.

4

MENGIKUTI SATU BERKAS CSV

Bag. 4.1 Buat berkas kecil yang dapat dipahami dengan mata

Buat `sensor-readings.csv` dengan empat baris berikut:

```
device,temperature,room
alpha,24.7,lab
beta,,office
alpha,24.7,lab
gamma,31.2,workshop
```

Kita sudah mengetahui rahasianya. Satu nilai suhu kosong. Baris alpha muncul dua kali. Ruang workshop lebih hangat daripada ruangan lain. Berkas mungil membuat kita mudah memeriksa apakah laporan berkata jujur.

Buka CSV Profiler, pilih berkas tersebut, lalu tekan Run Profiling. Biarkan `python/profiler_server.py` terbuka sementara layar progress bergerak.

Bag. 4.2 Route upload menerima amplopnya

Cari `@app.route("/upload")` di berkas Python. Flask memanggil fungsi di bawah baris itu ketika browser mengirim sebuah berkas.

Baca pemeriksaan awal sebagai pertanyaan biasa:

```
1 if "file" not in request.files:
2     return jsonify({"error": "No file provided"}), 400
3
4 if not f.filename.lower().endswith(".csv"):
5     return jsonify({"error": "Only CSV files are supported"}), 400
```

Jika amplop berisi CSV, Flask memberikan ID acak kepada pekerjaan tersebut lalu menyimpan salinan kerja. Nama berkas asli masuk ke catatan sesi. Nama berkas yang tersimpan memakai ID acak, sehingga dua orang dapat mengunggah `data.csv` tanpa berebut path yang sama.

Setelah itu, route menjalankan `run_profiling` pada background thread dan mengirim ID pekerjaan kepada browser.

Mengapa menjawab sebelum laporan selesai?

Profil lengkap dapat memakan waktu. Jika satu permintaan browser menunggu seluruh pekerjaan tanpa kabar, halaman terasa membeku. ID pekerjaan memberi browser sesuatu yang dapat diikuti sementara Python meneruskan pekerjaannya.

Bag. 4.3 Progress datang seperti catatan di bawah pintu

Setiap pekerjaan aktif memiliki antrean Python kecil. Profiler meletakkan pesan kemajuan ke dalamnya. Route `/progress/<job-id>` menunggu pesan tersebut lalu mengirimkannya kepada browser saat pesan tiba.

Aliran satu arah ini disebut server-sent events, biasa disingkat SSE. Gambarnya lebih berguna daripada namanya: Python menulis catatan pendek, lalu browser membacanya tanpa perlu mengulang pertanyaan yang sama terus-menerus.

```
1 def generate():
2     while True:
3         message = q.get()
4         if message is None:
5             break
6         yield message
```

Ketika antrean menerima `None`, aliran ditutup. Browser kemudian membuka laporan yang sudah selesai.

Bag. 4.4 Tempat laporan beristirahat

Flask menyimpan HTML laporan di `python/reports`. Di sebelahnya terdapat sebuah JSON sidecar. Anggap sidecar sebagai label pada map arsip: nama berkas asli, job ID, waktu pembuatan, alamat laporan, jumlah baris, dan jumlah kolom.

Previous Runs membaca label tersebut melalui `GET/sessions`. Refresh browser tidak menghapus sesi karena rincian penting berada di disk, tidak hanya di memori JavaScript.

Route berikut membentuk rak sesi kecil:

Metode	Route	Hasil yang diterima
GET	<code>/sessions</code>	Semua sesi laporan yang masih ditemukan Flask.
GET	<code>/session/<job-id></code>	Catatan untuk satu sesi tersimpan.
GET	<code>/report/<job-id></code>	Laporan HTML yang dihasilkan.
DELETE	<code>/reset/<job-id></code>	Penghapusan satu sesi beserta berkas kerjanya.
DELETE	<code>/reset-all</code>	Penghapusan seluruh sesi CSV lokal.

Bag. 4.5 Bersihkan salinan, lalu lihat lagi

Kembali ke berkas empat baris tadi. Pada panel cleaning, pilih penghapusan duplikat dan pengisian suhu kosong dengan median. Ketika pilihan diterapkan, browser mengirim deskripsi JSON kepada `/clean/<job-id>`.

Bentuk idenya seperti ini:

```
{
  "drop_duplicates": true,
  "missing": {
    "temperature": "fill_median"
  }
}
```

Pandas membuka data kerja saat ini, menerapkan pilihan tersebut, lalu menulis `<job-id>_cleaned.csv`. Upload asli tetap tersedia. Flask memperbarui catatan laporan dan memberikan alamat download kepada browser.

Buka CSV hasil pembersihan. Seharusnya ada tiga baris data dan tidak ada suhu kosong. Karena 24.7 muncul dua kali pada nilai awal, hasil pengisian median juga semestinya 24.7.

Bag. 4.6 Gunakan playground sebagai kertas coretan

Playground menjalankan satu gagasan Pandas kecil pada satu waktu. Mulailah dengan Preview, Head, Tail, Describe, atau Value Counts. Pilihan tersebut tidak mengubah data yang tersimpan.

Sebagai contoh, permintaan berikut meminta sepuluh baris dengan suhu terbesar:

```
{
  "operation": "sort_values",
  "column": "temperature",
  "limit": 10,
  "mutate": false
}
```

Kata `mutate` berarti “ubah data kerja.” Dashboard hanya mengirim `true` setelah Anda menekan tindakan perubahan secara tegas untuk operasi seperti rename, fill, drop, konversi tipe, atau penggantian isi sel.

Route membatasi jumlah baris antara 1 dan 500. Batas kecil ini mencegah preview raksasa memenuhi halaman karena salah ketik.

Bag. 4.7 Menambah fitur CSV tanpa mengganggu OTA

Letakkan operasi data baru di route Flask atau helper Python. Tambahkan kontrol yang sesuai ke `index.html` dan perilakunya ke `app.js`. Pertahankan permintaannya pada `API_BASE`.

Dengan aturan yang sama, fitur OTA tinggal di Axum dan `ota.js`. Kedua meja kerja berbagi meja penerima tamu, tetapi masing-masing memakai alatnya sendiri.

Berhenti sebentar sebelum lanjut

Jangan mengganti nama route CSV yang sudah ada ketika menambah fitur firmware. Perilaku frontend yang tersimpan sudah bergantung pada alamat tersebut. Tambahan kecil di sekitar alur yang bekerja lebih aman daripada penulisan ulang.

Coba dahulu sebelum lanjut

Profilkan berkas empat baris, hapus duplikatnya, isi suhu kosong, lalu download hasilnya. Refresh browser dan pulihkan sesi dari Previous Runs. Setelah itu, jelaskan mengapa laporan tetap ada walaupun memori JavaScript sudah dimuat ulang.

BAGIAN III

LAYANAN OTA RUST



Sekarang kita masuk ke server Rust. Satu ZIP firmware akan dibuka dengan hati-hati, lalu kita ikuti sampai berubah menjadi image aplikasi sungguhan beserta riwayat build-nya.

5

BERKENALAN DENGAN SERVER OTA RUST

Bag. 5.1 Baca main.rs seperti rutinitas membuka bengkel

Buka `ota-server/src/main.rs`. Sekilas berkas ini mungkin tampak resmi, padahal tugasnya mirip rutinitas membuka bengkel pada pagi hari. Nyalakan lampu. Buka lemari. Letakkan alat di meja. Buka pintu depan.

Versi Rust menjalankan rutinitas tersebut dalam urutan berikut:

1. Nyalakan logging agar terminal dapat menceritakan apa yang terjadi.
2. Baca pengaturan dari `.env`.
3. Buka basis data SQLite.
4. Siapkan builder firmware.
5. Siapkan coordinator yang mengawasi setiap build.
6. Masukkan bagian bersama itu ke `AppState`.
7. Hubungkan route HTTP.
8. Dengarkan alamat dan port yang sudah dikonfigurasi.

Berikut bagian tengah rutinitas tersebut dengan beberapa rincian dilipat:

```
1 let config = Arc::new(Config::load()?);
2 let storage = Storage::connect(&config.database_url()).await?;
3 let builder = Arc::new(LocalFirmwareBuilder::new(/* settings */));
4 let coordinator = Arc::new(BuildCoordinator::new(
5     storage.clone(),
6     builder,
7     config.project_dir.clone(),
8     config.build_dir.clone(),
9 ));
10
11 let state = AppState { config, storage, coordinator };
```

Kata `Arc` berarti beberapa permintaan dapat memegang referensi bersama menuju object layanan yang sama dengan aman. Anda belum perlu menguasainya. Perhatikan bentuk besarnya: konfigurasi, penyimpanan, dan proses build bertemu dalam satu bundel state. Route handler menerima bundel itu ketika membutuhkannya.

Bag. 5.2 Konfigurasi adalah halaman tata tertib rumah

Pindah ke `ota-server/src/config.rs`. Berkas ini membaca setiap pengaturan `OTA_...`, menyediakan nilai bawaan lokal, memeriksa isinya, dan menyelesaikan path folder dari root repositori.

Berkas yang sama membuat `firmware-projects`, `firmware-builds`, dan `data` jika belum ada. Karena itu, salinan proyek yang masih baru dapat menyala tanpa meminta Anda membuat folder tersebut dengan tangan.

Jika public URL tidak diisi, server mencoba mendeteksi alamat LAN yang masuk akal. Deteksi otomatis cukup nyaman pada jaringan rumah sederhana. `OTA_PUBLIC_BASE_URL` yang ditulis secara tegas lebih mudah dipahami ketika PC mempunyai Ethernet, Wi-Fi, virtual adapter, atau VPN sekaligus.

Ketika startup berhasil, terminal menampilkan dua sudut pandang:

```
OTA Server running
Local:
http://localhost:7000

LAN / public firmware URL:
http://192.168.1.5:7000
```

Baris local dipakai browser pada PC. Baris LAN adalah alamat yang dapat dijangkau ESP32.

Bag. 5.3 Route adalah label di meja penerima

Buka `ota-server/src/routes/mod.rs`. Berkas ini pendek karena pekerjaan utamanya memasang alamat dengan fungsi yang menanganinya.

Pekerjaan tersebut dikelompokkan ke tiga berkas di dekatnya:

`routes/firmware.rs` Menerima ZIP proyek dan menyajikan firmware yang telah selesai.

`routes/builds.rs` Memulai build dan mengembalikan riwayat atau log build.

`routes/devices.rs` Mengingat board, heartbeat, deployment, pemeriksaan update, dan kemajuan OTA.

Cari baris berikut:

```
.route("/api/ota/status", get(status))
```

Baca dari kiri ke kanan: ketika permintaan GET tiba pada alamat tersebut, panggil `status`. Baris route lain memakai tata bahasa yang sama.

Bag. 5.4 Mengapa browser diperbolehkan masuk

Halaman berasal dari port 5000 dan memanggil server pada port 7000. Browser menganggap keduanya sebagai origin berbeda. Karena itu, Axum membutuhkan aturan CORS yang menyebut origin dashboard yang dipercaya.

Daftar bawaan berisi:

```
http://localhost:5000
http://127.0.0.1:5000
```

Jika frontend disajikan dari alamat lain, tambahkan origin yang tepat ke `OTA_ALLOWED_ORIGINS`. Kesalahan CORS dapat muncul di browser walaupun URL status Axum dapat dibuka secara normal pada tab tersendiri.

Berhenti sebentar sebelum lanjut

Jaga daftar origin tetap sempit. Jika setiap website diizinkan memanggil builder firmware lokal, kemudahan kecil pada browser berubah menjadi pintu terbuka yang tidak diperlukan.

Bag. 5.5 Pesan kesalahan perlu berbicara dengan kalimat lengkap

Buka `ota-server/src/errors.rs`. Setiap kesalahan API biasa membawa status HTTP, kode pendek yang dapat dibaca mesin, dan pesan untuk manusia. Kesalahan juga dapat membawa rincian dan petunjuk.

```
{
  "success": false,
  "error": {
    "code": "INVALID_FIRMWARE_PROJECT",
    "message": "Managed application entrypoint was not found",
    "details": "src/main.rs"
  }
}
```

Frontend dapat menampilkan pesan tanpa menebak penyebabnya. Pengembang dapat memakai kode pendek dalam tes. Nomor status memberi kategori umum kepada client HTTP:

- 400** Permintaan tidak dapat dibaca.
- 404** Catatan yang disebut tidak ada.
- 409** Permintaan bertabrakan dengan keadaan saat ini, misalnya build aktif kedua.
- 413** Upload terlalu besar.
- 422** Permintaan dapat dibaca, tetapi melanggar aturan proyek.
- 500** Sesuatu di dalam server gagal secara tidak terduga.

Bag. 5.6 SQLite adalah buku catatan server

Berkas basis datanya adalah `data/ota.db`. Tabel tidak perlu dibuat dengan tangan. `storage.rs` melakukannya saat startup.

Setiap tabel menjawab pertanyaan yang berbeda:

Tabel	Pertanyaan yang dijawab
projects	ZIP mana yang diunggah, untuk target dan versi apa?
builds	Apa yang terjadi ketika proyek dicoba untuk dikompilasi?
firmware	Binary berhasil mana yang dapat diunduh perangkat?
devices	Board mana yang sudah memperkenalkan diri, dan apa laporan terakhirnya?
deployments	Firmware mana ditugaskan kepada board mana, dan sudah sejauh apa prosesnya?

Foreign key SQLite mengikat halaman-halaman itu. Berkas firmware yang masih dipakai perangkat atau deployment tidak dapat dihapus sembarangan. API mengembalikan conflict, sehingga tidak ada halaman

yang dicabut dari buku catatan sambil meninggalkan rujukan tanpa tujuan.

Coba dahulu sebelum lanjut

Buka `main.rs`, `config.rs`, `routes/mod.rs`, dan `storage.rs` pada empat tab. Di setiap tab, cari satu bagian yang cocok dengan peran bab ini: startup, pengaturan, alamat, dan tabel basis data.

6

APA YANG SEBENARNYA TERJADI PADA ZIP

Bag. 6.1 Mulai dari formulir upload

Pada halaman OTA Firmware, formulir meminta tiga hal: sebuah ZIP, sebuah board, dan sebuah versi. Browser memasukkannya ke permintaan multipart dengan nama field berikut:

Field	Contoh	Alasan server membutuhkannya
<code>project</code>	<code>sensor.zip</code>	Kode Rust dan berkas Cargo
<code>target</code>	<code>esp32s3</code>	Pilihan compiler dan chip
<code>version</code>	<code>1.0.1</code>	Nama rilis yang akan dilaporkan firmware baru

Axum memberi upload sebuah project ID acak. ID tersebut menjadi nama folder ekstraksi privat. Nama berkas yang diberikan browser tidak pernah diperbolehkan menentukan tempat penulisan server.

Bag. 6.2 Server memeriksa koper sebelum membukanya

ZIP mirip koper dengan daftar tujuan untuk setiap isinya. Entri nakal dapat mengaku bahwa berkasnya harus diletakkan di `../.. /somewhere-else`. Jika nama itu diekstrak tanpa pemeriksaan, arsip dapat kabur dari kamarnya. Serangan ini sering disebut ZIP slip.

Extractor menolak parent path, absolute path, symbolic link, isi lebih dari 10.000 entri, dan jumlah data hasil ekstraksi yang melampaui batas konfigurasi. Extractor juga mewajibkan adanya `Cargo.toml`.

Pemeriksaan tersebut mungkin terasa ketat ketika ZIP buatan sendiri ditolak. Penjaga ini melindungi semua folder lain di PC. Perbaiki susunan ZIP dan biarkan batas keamanannya tetap berdiri.

Bag. 6.3 Proyek memilih satu dari dua jalur

Setelah ekstraksi, `services/project.rs` mencari `ota-app.toml`.

Jika berkas itu ada, proyek diperlakukan sebagai *managed application*. Proyek memberikan perilaku produk, sedangkan runtime stabil memberikan Wi-Fi, OTA, dan rollback.

```
schema_version = 1
application_api_version = 2
entrypoint = "src/main.rs"
```

Jika berkas tersebut tidak ada, proyek menjadi *standalone firmware*. Axum membangunnya sebagaimana

proyek datang. Proyek standalone harus membawa client OTA sendiri jika ingin memperbarui dirinya lagi.

Managed mode lebih mudah untuk aplikasi pertama. Anda dapat menulis perilaku sensor tanpa menyalin kode jaringan dan partisi ke dalam setiap upload.

Bag. 6.4 Bagaimana dua berkas bernama `main.rs` hidup bersama

Bagian ini layak dibaca pelan-pelan.

Runtime stabil mempunyai executable biasa `fnmain()` di `examples/esp32-rust-ota-client/src/main.rs`. Ia memiliki proses boot, Wi-Fi, OTA, dan startup aplikasi.

ZIP Anda juga mempertahankan `src/main.rs` dengan bentuk yang terasa seperti program firmware biasa. Berkas tersebut mengekspos dua fungsi bebas:

```
1 pub fn setup(context: &mut AppContext) -> anyhow::Result<State>;
2 pub fn main(context: AppContext, state: State) -> anyhow::Result<()>;
```

Axum menyalin runtime stabil ke workspace hasil generate yang privat. Aplikasi ditempatkan di `applications/device-app`, lalu hanya manifest Cargo pada salinan itu yang disesuaikan. Di dalam salinan tersebut, Cargo memperlakukan `src/main.rs` aplikasi sebagai berkas sumber library. Runtime kemudian dapat memanggil `device_app::setup` dan `device_app::main`.

Berkas yang Anda upload tidak berubah. Rust menautkan kedua bagian menjadi satu image lengkap.

Gambaran sebuah dapur

Runtime stabil adalah dapur yang sudah memiliki listrik, air, dan aturan keselamatan. Upload Anda adalah resep yang dimasak hari ini. Hidangan akhir memerlukan keduanya. Resep baru tidak menumpuk dapur kedua di atas dapur pertama.

Bag. 6.5 Builder memeriksa alatnya

Runner perintah yang sebenarnya berada di `services/builder.rs`. Sebelum menyentuh proyek, ia menanyakan versi kepada tiga program:

```
1 cargo --version
2 rustc --version
3 espflash --version
```

Alat yang hilang menghasilkan catatan build gagal dengan pesan jelas. Axum tetap hidup, sehingga lingkungan dapat diperbaiki dan build baru dapat dicoba.

Builder berada di balik trait Rust kecil bernama `FirmwareBuilder`. Hari ini implementasinya menjalankan alat langsung pada PC. Kelak implementasi lain dapat menjalankannya di dalam container yang dikunci, sementara route API tetap sama.

Bag. 6.6 Cargo mengubah source menjadi ELF

Perintah build-nya adalah:

```
1 cargo build --release --target xtensa-esp32s3-espidf
```

Axum tidak menyusun kalimat shell dari teks upload. Ia menjalankan **cargo** lalu memberikan setiap argumen secara terpisah. Working directory-nya adalah workspace proyek yang sudah dijaga.

Selama Cargo bekerja, server membaca kedua output stream baris demi baris dan menyimpannya di catatan build. Karena itu, dashboard dapat menampilkan pesan compiler asli. Build lama yang gagal pun tetap memiliki log berguna setelah browser di-refresh.

Bag. 6.7 espflash menyiapkan image OTA

ELF dari Cargo berguna bagi alat pengembangan, sedangkan partisi OTA yang tidak aktif memerlukan image aplikasi ESP-IDF. Perintah berikutnya mengikuti bentuk ini:

```
1 espflash save-image --chip esp32s3 --format esp-idf \  
2 <ELF_FILE> firmware-sensor-v1.0.1-a8c4f2.bin
```

Builder secara sengaja meminta application image. Ia tidak membuat merged factory image yang ikut membawa bootloader dan data partisi. Bagian lebih besar tersebut dipasang ketika provisioning awal melalui USB.

Bag. 6.8 Sidik jari menutup paket

Server membaca berkas **.bin** yang selesai dan menghitung SHA-256. Anggap hash sebagai sidik jari panjang. Perangkat menghitung sidik jari yang sama selama download. Jika kedua string berbeda, byte sudah berubah atau berkas yang tiba bukan berkas yang diminta. Perangkat pun menolak mengaktifkannya.

Saat berhasil, SQLite menerima nama berkas, ukuran byte, target, versi, project ID, build ID, timestamp, dan hash. Dashboard kemudian menambahkan image ke tabel firmware.

Sidik jari berbeda dari tanda tangan

SHA-256 menangkap kerusakan data. Hash tidak memberi tahu perangkat siapa yang menyetujui firmware tersebut. Identitas penerbit pada sistem produksi membutuhkan signing yang nyata dan kunci yang dilindungi. Proyek ini menyisakan batas tersebut dengan jelas untuk pengembangan berikutnya.

Bag. 6.9 Baca status build sebagai cerita pendek

```
uploaded → queued → building → success  
                                ↘ failed
```

Uploaded berarti koper lolos pemeriksaan. Queued berarti catatan build sudah dibuat. Building berarti

alat sedang bekerja. Success berarti binary dan metadata benar-benar ada. Failed berarti log menyimpan alasan perjalanan berhenti.

Hanya satu build queued atau aktif yang diperbolehkan untuk proyek yang sama. Klik kedua menerima conflict, sehingga dua compiler tidak bertabrakan di folder kerja yang sama.

ZIP milik Anda tetap dapat menjalankan kode di PC

Cargo dapat menjalankan `build.rs`, procedural macro, dan build script milik dependensi. Ekstraksi aman menjaga path, tetapi proses kompilasi tetap menjalankan kode. Gunakan proyek yang dipercaya sampai builder berada di dalam sandbox yang layak.

Coba dahulu sebelum lanjut

Buka `examples/uploaded-firmware-basic`. Pastikan root ZIP-nya akan berisi `Cargo.toml`, `ota-app.toml`, dan `src/main.rs`. Upload sebagai versi 1.0.1, lalu perhatikan log. Toolchain lengkap harus menghasilkan binary sungguhan. Alat yang hilang harus menghasilkan kegagalan sungguhan. Kedua hasil itu sama-sama berkata jujur.

BAGIAN IV

FIRMWARE DI ESP32-S3

ESP32 akhirnya ikut bekerja. Mula-mula ia menerima fondasi yang andal melalui USB. Sesudah itu, kode aplikasi Anda dapat ikut masuk ke dalam setiap rilis OTA yang lengkap.

7

PROGRAM YANG TINGGAL DI DALAM ESP32

Bag. 7.1 Berikan fondasi yang dapat diandalkan kepada board

Pegang ESP32-S3 Anda sejenak. Sebelum OTA dapat membantu, board ini membutuhkan program yang tahu cara bergabung ke Wi-Fi, menemukan PC, dan menulis image baru dengan aman. Program pertama itu adalah runtime stabil di `examples/esp32-rust-ota-client`.

Kata *stabil* tidak berarti runtime tidak boleh berubah selamanya. Artinya, pembaruan produk sehari-hari tidak perlu menulis ulang seluruh saluran dasarnya. Runtime mengurus:

- startup dan log ESP-IDF;
- koneksi Wi-Fi;
- identitas perangkat dan heartbeat;
- pemeriksaan serta download update;
- dua slot aplikasi OTA;
- verifikasi hash, reboot, dan rollback;
- saat aplikasi upload Anda mulai dijalankan.

Aplikasi dapat mencurahkan perhatiannya pada sensor, motor, layar, atau pekerjaan produk lainnya.

Bag. 7.2 Perjalanan pertama melewati USB

Buka PowerShell di folder runtime. Isi nilai jaringan dan alamat server yang dapat dijangkau:

```
1 cd examples\esp32-rust-ota-client
2 $env:WIFI_SSID="your-ssid"
3 $env:WIFI_PASS="your-password"
4 $env:OTA_SERVER_URL="http://192.168.1.5:7000"
5 $env:DEVICE_NAME="Sensor node"
```

Periksa IP dengan teliti. Alamat tersebut milik PC yang menjalankan Axum.

Ketika board, kabel, dan tabel partisi sudah siap, perintah berikut membangun firmware, melakukan flash, lalu membuka monitor:

```
1 cargo +esp run --release --target xtensa-esp32s3-espidf
```

Cargo runner yang dikonfigurasi akan memanggil `espflash flash -monitor`. Flash pertama dapat memasang bootloader dan susunan partisi bersama aplikasi.

Berhenti dan kenali board yang terhubung

Flashing menulis ke hardware fisik. Pastikan board tersebut benar-benar ESP32-S3, kabel USB dapat membawa data, ukuran flash cocok dengan rencana partisi, dan port serial yang dipilih memang milik board ini.

Saat runtime boot, terminal serial akan menampilkan kegiatan Wi-Fi. Setelah memperoleh alamat IP, perangkat mendaftarkan ke Axum. Refresh halaman Devices pada dashboard dan cari nama yang tadi Anda berikan.

Bag. 7.3 Baca fn main yang sebenarnya seperti sebuah cerita

Buka `examples/esp32-rust-ota-client/src/main.rs`. Fungsi utamanya dimulai di sini:

```
fn main() -> Result<()> {
    esp_idf_svc::sys::link_patches();
    EspLogger::initialize_default();
    // platform setup continues...
}
```

Turuni berkas itu perlahan. Runtime mengambil peripheral board, memisahkan modem, menjalankan system event loop dan NVS, lalu membuat `WifiManager`. Device ID diturunkan dari station MAC kecuali Anda sudah memberikannya sendiri.

Berikutnya, runtime membuat `AppContext`. Inilah tas yang diserahkan kepada aplikasi. Sebelum aplikasi dipanggil, runtime bertanya kepada ESP-IDF apakah image saat ini masih baru dan menunggu validasi.

Lalu terjadilah serah terima:

```
1 let application_state = device_app::setup(&mut context)?;
2
3 if unverified_image {
4     mark_running_image_valid()?;
5 }
6
7 // Start the OTA supervisor in its own thread.
8 device_app::main(context, application_state)
```

Kode asli memiliki penanganan kesalahan yang lebih lengkap. Bentuk yang perlu diingat adalah: siapkan platform, uji setup aplikasi, setuju slot baru, biarkan OTA tetap hidup, lalu masuki main loop aplikasi.

Bag. 7.4 Buka crate yang lebih kecil

Runtime dibagi agar tiap bagian muat di kepala saat sedang dipelajari:

crates/application-api Mendefinisikan hal-hal yang boleh diterima aplikasi.

crates/wifi-manager Memiliki modem dan terus mencoba ketika jaringan terputus.

crates/ota-runtime Berbicara dengan Axum dan menulis update ke slot yang tidak aktif.

applications/default-app Menyediakan perilaku kecil untuk flash USB pertama.

Anda dapat membuka satu crate tanpa membaca semuanya sekaligus. Pemisahan ini sangat membantu ketika masalah Wi-Fi dan masalah sensor kebetulan muncul pada hari yang sama.

Bag. 7.5 AppContext adalah tas serah terima

Aplikasi menerima bentuk besar berikut:

```
1 pub struct AppContext {
2     pub device: DeviceInfo,
3     pub network: NetworkHandle,
4     pub peripherals: ApplicationPeripherals,
5 }
```

`device` memberi tahu ID, nama ramah, chip, dan versi firmware. `network` membiarkan aplikasi menanyakan apakah Wi-Fi tersambung dan IP mana yang terakhir dilaporkan. `peripherals` membawa pin serta controller yang tersedia.

Setiap item hardware dibungkus dalam `Option`. Ketika fungsi `setup` mengambil I2C0, option tersebut menjadi kosong. Kode sekarang mempunyai catatan yang terlihat bahwa satu bagian aplikasi telah memiliki controller itu.

Modem memang tidak dimasukkan ke tas. `wifi-manager` menyimpannya agar heartbeat dan polling OTA terus berjalan selama aplikasi bekerja.

Apakah Wi-Fi manager akan menjadi masalah?

Wi-Fi manager justru membantu ketika kepemilikan berada di satu tempat. Manager saat ini sudah memiliki modem, event loop, layanan Wi-Fi yang memakai NVS, dan reconnect loop. Setup access point atau captive portal di masa depan dapat tumbuh di sana. Aplikasi upload tetap membaca network handle yang sama.

Bag. 7.6 Flash baru tidak menumpuk kode lama

Bayangkan dua laci berlabel `ota_0` dan `ota_1`. Board menjalankan isi satu laci sementara updater menulis image lengkap ke laci lainnya. Setelah verifikasi dan reboot, kedua laci bertukar peran.

Image yang baru ditulis berisi runtime dan aplikasi pilihan sebagai satu kesatuan. Image tersebut mengganti isi slot yang tidak aktif dari awal sampai akhir. Struct, fungsi, dan library lama tidak tetap dimuat di sampling program baru.

Laci yang sebelumnya aktif masih disimpan untuk rollback sampai image baru membuktikan dirinya dapat berjalan. Pada rilis berikutnya, ESP-IDF dapat menimpa laci yang lebih tua. PC boleh menyimpan riwayat firmware panjang di SQLite, sementara board tetap hanya mempunyai dua slot aplikasi.

Coba dahulu sebelum lanjut

Sebelum menghubungkan hardware, cari `fn main` milik runtime, panggilan ke `device_app::setup`, panggilan ke `device_app::main`, dan tempat modem dipisahkan dari peripheral aplikasi. Ikuti keempatnya dengan jari dalam urutan tersebut.

8

MENULIS FIRMWARE UPLOAD PERTAMA

Bag. 8.1 Salin contoh kecil sebelum memulai dari halaman kosong

Buka `examples/uploaded-firmware-basic`. Isinya hanya bagian yang diharapkan server:

```
Cargo.toml
ota-app.toml
src/
  main.rs
```

Salin folder ini ke tempat yang aman, lalu berikan nama baru kepada package Cargo. Memulai dari contoh yang bekerja membuat Anda dapat mengubah satu gagasan setiap kali.

Descriptor memberi tahu Axum cara memperlakukan proyek:

```
schema_version = 1
application_api_version = 2
entrypoint = "src/main.rs"
```

Biarkan nilai-nilai tersebut tetap seperti itu untuk aplikasi pertama.

Bag. 8.2 Program Anda tetap terasa seperti `main.rs`

Buka `src/main.rs`. Tidak ada application class dan tidak ada framework trait yang wajib diimplementasikan. Anda menulis dua fungsi public.

```
1 use anyhow::Result;
2 use firmware_app_api::AppContext;
3
4 pub fn setup(context: &mut AppContext) -> Result<u64> {
5     log::info!("Hello from {}", context.device.device_id);
6     Ok(0)
7 }
8
9 pub fn main(context: AppContext, mut count: u64) -> Result<()> {
10     loop {
11         count += 1;
12         log::info!(
13             "count={count}, wifi={}",
14             context.network.is_connected()
15         );
16         std::thread::sleep(std::time::Duration::from_secs(10));
17     }
18 }
```

setup berjalan satu kali setelah runtime menyiapkan board. main adalah loop aplikasi yang berumur panjang. Entrypoint stabil memanggil keduanya untuk Anda.

Ubah pesan hello. Satu baris itu sudah cukup untuk build pertama yang terasa milik Anda sendiri.

Bag. 8.3 Nilai antara setup dan main adalah kotak perkakas

Contoh mengembalikan sebuah angka dari setup, lalu menerimanya sebagai count di main. Angka itu adalah state aplikasi.

State milik Anda dapat sesederhana () jika tidak ada apa pun yang perlu dibawa. State juga dapat berupa struct yang menyimpan driver sensor, counter, buffer, atau keadaan layar.

```
1 pub struct State {
2     sample_number: u64,
3     alarm_enabled: bool,
4 }
5
6 pub fn setup(context: &mut AppContext) -> Result<State> {
7     log::info!("Preparing peripherals for {}", context.device.name);
8     Ok(State {
9         sample_number: 0,
10        alarm_enabled: true,
11    })
12 }
13
14 pub fn main(context: AppContext, mut state: State) -> Result<()> {
15     loop {
16         state.sample_number += 1;
17         // Read the sensor and do your real work here.
18     }
19 }
```

State ini hanyalah data Rust biasa. Sistem OTA tidak meminta bentuk class tertentu.

Bag. 8.4 Ambil peripheral saat setup

Misalkan aplikasi membutuhkan I2C0 dan beberapa pin. Bagian awalnya dapat berbentuk seperti ini:

```
1 use anyhow::{Context, Result};
2
3 pub fn setup(context: &mut AppContext) -> Result<State> {
4     let pins = context.peripherals.pins.take()
5         .context("GPIO pins were already taken")?;
6     let i2c0 = context.peripherals.i2c0.take()
7         .context("I2C0 was already taken")?;
8
9     // Construct the ESP-IDF I2C driver and sensor here.
10    // Return the finished driver inside State.
11 }
```

Panggilan `take()` memindahkan sumber daya tersebut ke kode Anda. Jika setup menerima sumber daya yang sudah tidak ada, ia mengembalikan kesalahan yang dapat dibaca. Pada image OTA yang baru, kesalahan itu dapat memicu rollback.

Tempatkan pembuatan driver yang mungkin gagal dan pemeriksaan startup singkat di setup. Jangan biarkan setup berputar selamanya. Runtime memerlukan jawaban sukses atau gagal yang jelas sebelum menyetujui slot baru.

Berhenti sebentar sebelum lanjut

Kembalikan state yang dimiliki aplikasi dari setup. Referensi yang dipinjam dari `AppContext` tidak dapat hidup dengan aman setelah panggilan setup mutable berakhir dan context dipindahkan ke main. Driver ESP-IDF nyata mungkin memerlukan lifetime dan type alias yang teliti. Ikuti contoh crate driver untuk chip serta pin yang Anda gunakan.

Bag. 8.5 Tambahkan library Cargo biasa

Ya, aplikasi upload dapat memakai library. Buka `examples/uploaded-firmware-with-libs/Cargo.toml`:

```
[dependencies]
anyhow = "1"
firmware-app-api = "0.1"
heapless = "0.8"
log = "0.4"
serde = { version = "1", features = ["derive"] }
serde_json = "1"
```

Contoh tersebut menyimpan sampel di dalam antrean berkapasitas tetap dan mengubah telemetry menjadi JSON. Driver sensor, helper `embedded-hal`, koleksi ringkas, dan crate serialisasi dapat dipakai dengan cara serupa.

Library harus mendukung `xtensa-esp32s3-espidf`. Crate desktop yang menganggap mouse, filesystem, atau sistem operasi penuh selalu tersedia mungkin sulit dikompilasi atau memang tidak masuk akal di board. Baca contoh dan daftar feature milik crate tersebut.

Jika memakai local path dependency, masukkan seluruh foldernya ke dalam ZIP. Path harus tetap berada di bawah root proyek upload. Axum menolak path yang mencoba keluar dari arsip.

Mengapa firmware-app-api tampak seperti dependensi biasa

Manifest upload memakai `firmware-app-api="0.1"` sebagai placeholder yang mudah dibawa. Di dalam salinan build privat, Axum mengarahkan nama itu ke API crate yang cocok di runtime stabil. Manifest asli Anda dibiarkan tetap utuh.

Bag. 8.6 Berikan versi yang berguna kepada rilis

Versi yang diketik di dashboard menjadi versi firmware yang dikompilasi ke managed runtime dan disimpan di SQLite. Mulailah dengan urutan tenang seperti 1.0.0, 1.0.1, lalu 1.1.0.

Deployment memilih firmware ID tertentu. Server saat ini memeriksa apakah desired version sama dengan current version perangkat. Server belum mengurutkan semantic version dan memilih rilis secara otomatis.

Bag. 8.7 Zip isinya, jangan rak luarnya

Di Windows PowerShell:

```
1 Compress-Archive \  
2   -Path examples\uploaded-firmware-basic\* \  
3   -DestinationPath uploaded-firmware-basic.zip
```

Di Linux atau macOS:

```
1 cd examples/uploaded-firmware-basic  
2 zip -r ../../uploaded-firmware-basic.zip \  
3   Cargo.toml ota-app.toml src README.md
```

Buka ZIP dan lihat tingkat pertamanya. `Cargo.toml`, `ota-app.toml`, dan `src` harus langsung terlihat. Upload ZIP itu di OTA Firmware. Perangkat kelak menerima `.bin`, tidak pernah ZIP ini.

Coba dahulu sebelum lanjut

Buat salinan contoh basic. Ubah nama package dan pesan hello. Biarkan setup mengembalikan state struct kecil buatan Anda. Zip isinya lalu upload sebagai versi 1.0.1. Respons upload seharusnya menyebut `managed_application`, API versi 2, dan `src/main.rs`.

9

MELIHAT PERJALANAN SEBUAH UPDATE OTA

Bag. 9.1 Dashboard meninggalkan pesan untuk perangkat

Menekan Deploy OTA tidak memaksa sambungan masuk ke ESP32. Dashboard memberi tahu Axum firmware mana yang seharusnya diinginkan perangkat. Axum menyimpan pilihan itu dengan status pending.

ESP32 memeriksa server menurut jadwalnya sendiri. Pada pemeriksaan berikutnya, Axum menyerahkan pesan tersebut. Inilah pull-based update: perangkat memulai setiap percakapan jaringan.

```
Anda menekan Deploy OTA
↓
Axum mencatat firmware yang diinginkan
↓
ESP32 bertanya apakah ada update
download → periksa → pasang → reboot
↓
runtime baru melaporkan versinya
```

Board yang sedang tidur atau sementara offline dapat mengambil pesan itu nanti. PC tidak perlu mengetahui cara membuka koneksi menembus firewall board atau alamatnya yang berubah.

Bag. 9.2 Perangkat memperkenalkan diri

Setelah Wi-Fi tersambung, runtime mengirim registrasi seperti ini:

```
{
  "device_id": "ESP32-S3-A1B2C3D4E5F6",
  "name": "Sensor node",
  "chip": "esp32s3",
  "current_version": "1.0.0",
  "ip_address": "192.168.1.24"
}
```

Permintaan tersebut membuat kartu pada halaman Devices. Server menerima device ID rapi yang tersusun dari huruf, angka, titik, dash, atau underscore.

Runtime kemudian mengirim heartbeat pada setiap siklus. Heartbeat adalah pesan singkat “saya masih di sini” yang membawa versi, status, dan IP terkini. Axum memperbarui `last_seen` setiap kali pesan tiba.

Kartu lama dapat tetap terlihat setelah board kehilangan daya. Periksa `last_seen` sebelum mempercayai tulisan Online.

Bag. 9.3 Pemeriksaan update mengembalikan manifest kecil

Saat tidak ada firmware yang ditugaskan, perangkat menerima:

```
{"update_available": false}
```

Saat versi 1.0.1 menunggu, perangkat menerima deskripsi binary yang tepat:

```
{
  "update_available": true,
  "firmware_id": "c53b...",
  "version": "1.0.1",
  "target": "esp32s3",
  "size": 945332,
  "sha256": "f2534c...",
  "download_url": "http://192.168.1.5:7000/api/ota/firmware/c53b.../download"
}
```

Perangkat memeriksa target, ukuran, bentuk hash, dan URL sebelum membuka OTA writer. Server juga mencegah firmware ESP32-S3 ditugaskan kepada chip lain.

Bag. 9.4 Binary berpindah satu cangkir setiap kali

Ukuran image firmware dapat jauh lebih besar daripada RAM kosong pada perangkat kecil. Karena itu, client memakai buffer 4 KiB.

Bayangkan memindahkan air dari sebuah drum memakai satu cangkir. Baca satu cangkir dari HTTP. Tuangkan ke slot flash yang tidak aktif. Masukkan byte yang sama ke perhitungan SHA-256 yang sedang berjalan. Ulangi sampai respons selesai. Seluruh isi drum tidak pernah perlu ditampung di memori.

Saat byte datang, runtime melaporkan kemajuan download kira-kira setiap lima persen. Di ujung proses, runtime memeriksa jumlah byte keseluruhan dan membandingkan sidik jari akhir dengan nilai dari server.

Jika ukuran atau hash berbeda, writer dibatalkan. Aplikasi yang sedang berjalan tetap tidak tersentuh.

Bag. 9.5 Perangkat menceritakan jalannya update

Status callback membuat dashboard dapat mengikuti proses:

```
{"status": "downloading", "progress": 35}
{"status": "verifying", "progress": 100}
{"status": "installing", "progress": 100}
{"status": "rebooting", "progress": 100}
```

Kegagalan membawa alasannya:

```
{
  "status": "failed",
```

```
"error": "SHA256 mismatch"
}
```

Setelah reboot, image baru menjalankan setup. Jika setup berhasil, runtime menandai slot sebagai valid lalu melaporkan:

```
{
  "status": "success",
  "progress": 100,
  "current_version": "1.0.1"
}
```

Axum mengosongkan desired firmware. Rilis yang sama tidak akan terus ditawarkan pada setiap polling.

Bag. 9.6 Lihat isi tabel partisi dua laci

Contoh untuk flash 4 MiB yang disediakan adalah:

#	Name,	Type,	SubType,	Offset,	Size
nvs,	data,	nvs,		0x9000,	0x6000
otadata,	data,	ota,		0xf000,	0x2000
phy_init,	data,	phy,		0x11000,	0x1000
ota_0,	app,	ota_0,		0x20000,	0x1e0000
ota_1,	app,	ota_1,		0x200000,	0x1e0000

otadata mengingat slot aplikasi mana yang perlu di-boot. ota_0 dan ota_1 adalah dua lacinya. ESP-IDF memilih laci yang tidak aktif. Kode aplikasi tidak boleh mengganggu satu slot selalu menjadi pemenang.

Bag. 9.7 Rollback memberi masa percobaan kepada image baru

Contoh mengaktifkan CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE=y. Ketika image OTA boot untuk pertama kali, slot tersebut belum terverifikasi.

Runtime memanggil fungsi setup milik aplikasi. Setup yang berhasil membuat slot ditandai valid. Setup yang gagal membuatnya ditandai invalid dan meminta reboot. Bootloader yang memahami rollback kemudian dapat kembali ke slot valid sebelumnya.

Jaring pengaman ini bergantung pada kerja sama tabel partisi, konfigurasi bootloader, dan panggilan validasi aplikasi. Ujilah dengan setup yang sengaja gagal pada hardware Anda sebelum mengandalkannya.

Berhenti sebentar sebelum lanjut

SHA-256 dan HTTP biasa cukup untuk pemeriksaan kerusakan data pada LAN pengembangan yang dipercaya. Perangkat produksi juga membutuhkan rencana keaslian, misalnya signed image, kunci yang dilindungi, Secure Boot, dan kebijakan transport yang dirancang dengan sengaja.

Coba dahulu sebelum lanjut

Ceritakan perjalanan update tanpa menyebut nama endpoint: tinggalkan pesan, biarkan board bertanya, pindahkan image dengan cangkir kecil, bandingkan sidik jari, tukar laci, uji startup, lalu

laporkan versi baru. Setelah itu, cocokkan setiap langkah dengan kode atau layar dashboard.

BAGIAN V

LATIHAN DAN PENGEMBANGAN LANJUTAN



Bagian terakhir adalah meja praktik. Anda akan menjalankan alur lengkap, belajar membaca kegagalan yang umum, dan merancang pengembangan baru tanpa membongkar bagian yang sudah bekerja.

10

MENYATUKAN SELURUH ALUR

Bag. 10.1 Kosongkan sedikit ruang di meja

Sekarang waktunya menjalankan seluruh perjalanan. Siapkan PC, ESP32-S3, kabel USB data, dan informasi Wi-Fi. Buka satu terminal untuk Flask dan satu terminal untuk Axum. Sisakan ruang bagi terminal ketiga ketika serial monitor muncul.

Tidak ada hadiah untuk orang yang paling terburu-buru. Di akhir setiap tahap, carilah bukti kecil yang disebutkan pada halaman ini. Jika buktinya belum terlihat, tetaplah di tahap tersebut.

Bag. 10.2 Tahap satu: bangunkan kedua layanan PC

Buka `.env`. Periksa alamat LAN PC, nama Wi-Fi, kata sandi Wi-Fi, dan nama perangkat. Setelah itu, jalankan proyek:

```
# Windows
.\run-dev.bat

# Linux or macOS
./run-dev.sh
```

Buka dua alamat ini:

```
http://localhost:5000
http://localhost:7000/api/ota/status
```

Yang perlu dilihat: tab pertama menampilkan dashboard. Tab kedua mengatakan layanan OTA online dan mencetak public URL dengan alamat LAN yang benar.

Bag. 10.3 Tahap dua: periksa kesehatan jalur CSV lama

Upload `sensor-readings.csv` kecil dari Bab 4. Perhatikan progress, buka laporan, lalu jalankan preview di playground. Kunjungi Previous Runs dan pastikan sesinya tercantum.

Yang perlu dilihat: berpindah ke OTA Firmware lalu kembali lagi tidak menghilangkan sesi CSV. Bukti kecil ini menunjukkan bahwa halaman baru tetap menghormati pekerjaan Flask yang sudah ada.

Bag. 10.4 Tahap tiga: bungkus aplikasi basic

Di Windows PowerShell:

```
1 Compress-Archive \
2   -Path examples\uploaded-firmware-basic\* \
3   -DestinationPath uploaded-firmware-basic.zip
```

Buka ZIP tersebut. Tingkat pertamanya harus memperlihatkan `Cargo.toml`, `ota-app.toml`, dan `src`. Masuk ke OTA Firmware. Pilih ZIP, pilih ESP32-S3, ketik versi 1.0.1, lalu tekan Upload & Build.

Yang perlu dilihat: badge bergerak dari uploading menuju queued dan building. Terminal dimulai dengan versi alat dan mode managed application. Baris compiler yang sebenarnya akan menyusul.

Jika build gagal, simpan catatannya. Cari kesalahan pertama yang berguna dan perbaiki kondisi tersebut. Jangan menciptakan sukses palsu dengan mengedit SQLite atau meletakkan binary acak di `firmware-builds`.

Bag. 10.5 Tahap empat: periksa hasil build yang berhasil

Ketika badge mencapai success, lihat tabel firmware. Baris baru seharusnya berisi versi 1.0.1, target ESP32-S3, nama berkas, ukuran byte, SHA-256, waktu, dan status Ready.

Di balik layar, Anda akan menemukan:

- upload yang diekstrak di bawah `firmware-projects`;
- generated managed workspace di dalam folder proyek tersebut;
- application binary di bawah `firmware-builds`;
- baris project, build, dan firmware yang cocok di `data/ota.db`.

Tekan Download satu kali. Jumlah byte hasil download harus sama dengan angka pada tabel.

Bag. 10.6 Tahap lima: berikan program dasar pertama kepada board

Di sinilah kita menyentuh batas fisik. Tinjau `examples/esp32-rust-ota-client/partitions.csv`, lalu bandingkan dengan ukuran flash board. Hubungkan board memakai kabel data.

Atur nilai runtime, kemudian lakukan flash:

```
1 cd examples\esp32-rust-ota-client
2 $env:WIFI_SSID="your-ssid"
3 $env:WIFI_PASS="your-password"
4 $env:OTA_SERVER_URL="http://192.168.1.5:7000"
5 $env:DEVICE_NAME="Tutorial sensor"
6 cargo +esp run --release --target xtensa-esp32s3-espidf
```

Ganti contoh IP sebelum menekan Enter.

Berhenti sebentar sebelum lanjut

Perintah ini menulis ke board yang terhubung. Pastikan chip, port serial, ukuran flash, tabel partisi, dan catu daya stabil. Jangan memakai ulang tabel partisi contoh pada produk yang sudah berisi data sebelum seluruh data partition-nya disesuaikan.

Yang perlu dilihat: terminal serial menunjukkan startup dan koneksi Wi-Fi. Tidak lama kemudian, board muncul pada halaman Devices bersama versi dan alamat IP saat ini.

Bag. 10.7 Tahap enam: tinggalkan pesan update

Buka Devices, lalu tekan Deploy OTA untuk board baru. Pilih baris firmware versi 1.0.1.

Kartu pertama-tama menampilkan Pending. Runtime memeriksa setiap tiga puluh detik, jadi berikan sedikit waktu. Sesudah itu, perhatikan status Downloading, Verifying, Installing, dan Rebooting.

Serial monitor mungkin menghilang sebentar saat restart. Buka lagi jika diperlukan. Image baru menjalankan setup aplikasi, menandai slot sebagai valid, lalu memulai main loop yang diunggah.

Yang perlu dilihat: kartu perangkat mencapai Success dan current version berubah menjadi 1.0.1. Output serial memuat pesan siklus dari aplikasi basic.

Bag. 10.8 Tahap tujuh: ubah sesuatu yang mudah dikenali

Salin aplikasi basic. Ganti baris log dengan pesan yang langsung Anda kenali, misalnya nama ruangan atau sensor. Build sebagai 1.0.2, kemudian deploy.

Setelah reboot, cari pesan baru di serial monitor. Inilah momen yang menyenangkan: source dari ZIP Anda sedang berjalan di dalam image firmware lengkap yang dibangun oleh PC.

Untuk mencoba dependensi, gunakan `examples/uploaded-firmware-with-libs`. Contoh itu sudah memperlihatkan `heapless`, `Serde`, dan `JSON`.

Bag. 10.9 Tuliskan bukti selagi masih segar

Catatan lab singkat dapat menyelamatkan satu sore pada percobaan berikutnya. Tulis build ID, firmware ID, versi, ukuran byte, hash, device ID, versi lama dan baru, model board, versi alat, serta pesan serial terakhir. Jika suatu tahap membutuhkan perbaikan, sertakan kesalahan asli dan solusi yang benar-benar bekerja.

Seluruh perjalanan

Lab selesai ketika laporan CSV masih bekerja, build dengan toolchain asli menghasilkan firmware, perangkat yang telah diprovisi secara fisik mendaftar, deployment berubah menjadi update manifest, perangkat melakukan streaming dan verifikasi binary, lalu image setelah reboot melaporkan versi 1.0.1 atau yang lebih baru.

KETIKA SESUATU TIDAK BERJALAN

Bag. 11.1 Duduk sebentar di langkah terakhir yang berhasil

Sebuah kesalahan lebih mudah dihadapi setelah diberi alamat. Ajukan satu pertanyaan: langkah terakhir mana yang benar-benar berhasil?

Jika dashboard belum pernah terbuka, tetap periksa Flask. Jika CSV bekerja tetapi lampu OTA gelap, periksa Axum. Jika Axum menerima ZIP lalu Cargo gagal, baca build log. Jika PC dapat mengunduh binary tetapi board tidak dapat, periksa jaringan di antara keduanya.

Kebiasaan ini memakan sepuluh detik dan sering menghemat satu jam.

Bag. 11.2 Halaman dashboard tidak dapat dibuka

Lihat terminal Flask. Jika tertulis nama paket yang hilang, pasang `python/requirements.txt` ke dalam `python/.venv`. Setelah itu, coba:

```
http://127.0.0.1:5000/
```

Jika alamat tersebut terbuka, Flask sudah menyajikan halaman. Jika halaman tidak memiliki style atau tombol tidak bereaksi, buka `/style.css` dan `/app.js` pada port yang sama untuk memastikan kedua berkas ikut disajikan.

Jangan membuka `index.html` dengan klik ganda. Halaman `file:///` yang dihasilkan hidup di dunia browser berbeda dari dashboard yang disajikan Flask.

Bag. 11.3 CSV bekerja dan OTA mengatakan offline

Buka:

```
http://localhost:7000/api/ota/status
```

Jika gagal, baca terminal Axum. Cargo mungkin masih mengompilasi. Konfigurasi mungkin berisi port atau URL yang salah. Program lain mungkin sudah memakai port 7000.

Jika status dapat dibuka di tab tersendiri tetapi permintaan dashboard diblokir, buka panel Network di browser dan cari tulisan CORS. Bandingkan origin dashboard yang tepat dengan `OTA_ALLOWED_ORIGINS`.

Bag. 11.4 ZIP ditolak di depan pintu

Baca isi error sebelum membungkus ulang apa pun. Biasanya pesannya menunjuk salah satu hal berikut:

- berkas yang dipilih bukan ZIP;
- `Cargo.toml` tidak ada;
- `ota-app.toml` memakai versi API lama;
- `src/main.rs` hilang atau entrypoint menunjuk keluar proyek;
- local dependency mencoba keluar dari arsip;
- arsip memuat symlink atau path yang tidak aman;
- ukuran terkode atau hasil ekstraksi terlalu besar.

Buka ZIP dan lihat tingkat pertamanya. Susunan managed application yang bersih adalah `Cargo.toml`, `ota-app.toml`, dan `src`.

Bag. 11.5 Build berhenti sebelum kode Anda muncul

Baris log pertama menguji `cargo`, `rustc`, dan `esptool`. Jika satu alat hilang, jalankan perintah `--version` miliknya pada lingkungan terminal yang sama dengan Axum.

Setelah memasang alat, tutup dan nyalakan ulang terminal OTA. Sebuah process menyimpan lingkungan yang dimilikinya ketika pertama kali dijalankan.

Di Windows, pesan `link.exe not found` menunjuk ke Visual Studio Build Tools dengan Desktop development with C++. Toolchain Espressif sendiri tidak menyediakan host linker tersebut.

Bag. 11.6 Rust memberi satu halaman error compiler

Gulir ke kesalahan jelas pertama yang mencantumkan source path dan nomor baris. Kesalahan berikutnya sering tumbuh dari kesalahan pertama itu.

Untuk aplikasi upload, periksa bentuk serah terimanya:

```
pub fn setup(context: &mut AppContext) -> anyhow::Result<State>
pub fn main(context: AppContext, state: State) -> anyhow::Result<()>
```

Tipe state yang dikembalikan `setup` harus sama dengan parameter kedua pada `main`. Ambil hardware dari `context.peripherals`. Pastikan library baru mendukung target ESP32-S3 ESP-IDF.

Lakukan satu koreksi, lalu build lagi. Mengubah lima hal sekaligus akan menyembunyikan perubahan mana yang sebenarnya berpengaruh.

Bag. 11.7 Cargo berhasil dan pembuatan image gagal

Cari ELF path di dalam log. Baris berikutnya seharusnya memperlihatkan tahap `esptool save-image`. Jalankan `esptool --version` di terminal Axum dan pastikan perintah yang diharapkan tersedia.

Server hanya menerima application image yang tidak kosong di `firmware-builds`. Baris Cargo sukses tidak dapat menggantikan berkas `.bin` yang hilang.

Bag. 11.8 PC melihat firmware, board tidak melihatnya

Perhatikan `public_base_url` pada respons status. Nilainya harus berisi IP LAN milik PC. Coba buka URL LAN itu dari ponsel atau komputer lain pada Wi-Fi yang sama.

Jika perangkat kedua tidak dapat membukanya, periksa firewall PC, network profile, rute VPN, dan client isolation pada access point. Jika perangkat kedua dapat membukanya, bandingkan dengan URL yang benar-benar dikompilasi ke runtime ESP32.

Bag. 11.9 Kartu perangkat tidak pernah muncul

Dashboard tidak mencari perangkat USB secara otomatis. Registrasi harus dikirim oleh program yang berjalan di ESP32.

Buka serial monitor. Cari koneksi Wi-Fi terlebih dahulu, lalu cari registration POST. Pastikan board menerima SSID, password, dan LAN server URL yang benar ketika dibangun.

Jika provisioning Wi-Fi belum pernah dilakukan, kembali ke flash USB pertama di Bab 7.

Bag. 11.10 Kartu tetap berada pada Pending

Pending berarti Axum sudah menyimpan deployment. Server belum menerima kabar selesai dari perangkat.

Periksa `last_seen`. Runtime biasanya melakukan polling setiap tiga puluh detik. Pastikan desired version berbeda dari current version dan chip perangkat cocok dengan target firmware. Terminal serial semestinya memperlihatkan pemeriksaan update berikutnya atau kesalahan jaringannya.

Bag. 11.11 Ukuran atau SHA-256 tidak cocok

Runtime membatalkan writer pada slot yang tidak aktif. Itulah respons yang aman. Size mismatch mengarah pada download terpotong atau metadata yang salah. Hash mismatch berarti byte yang diterima tidak sama dengan artifact yang dicatat Axum.

Download firmware yang sama pada PC, periksa ukuran berkas, lalu bandingkan hash-nya dengan metadata. Jangan membuang pemeriksaan di sisi perangkat hanya agar update terus berjalan.

Bag. 11.12 Binary sudah terlalu besar untuk lacinya

Slot OTA contoh masing-masing berukuran 0x1e0000 byte. Bandingkan ukuran binary dengan ukuran partisi nyata. Kurangi kode atau dependensi terlebih dahulu. Slot yang lebih besar membutuhkan revisi tabel partisi dengan teliti dan biasanya diikuti provisioning serial baru.

Bag. 11.13 Delete menjawab 409 Conflict

Firmware masih terhubung ke desired state perangkat atau riwayat deployment. Penolakan itu melindungi hubungan antarcatatan. Jangan menghapus binary dengan tangan selama SQLite masih menunjuk kepadanya.

Bag. 11.14 Di mana pesan terminal ESP32?

Pesan tersebut masih berada di serial monitor. Build History menampilkan output compiler dari PC. Devices menampilkan heartbeat dan status OTA. Kode dashboard saat ini belum melakukan streaming log dari board.

Gunakan `espflashmonitor` untuk saat ini. Bab berikutnya menunjukkan bagaimana serial bridge atau network logger dapat membawa pesan tersebut ke panel dashboard yang diberi label jelas.

Coba dahulu sebelum lanjut

Ambil satu kegagalan nyata dari terminal. Tulis langkah terakhir yang berhasil, batas yang gagal, tes terkecil untuk batas itu, dan satu perubahan yang akan dicoba. Simpan pesan kesalahan asli di samping hasil percobaan.

ARAH PENGEMBANGAN BERIKUTNYA

Bag. 12.1 Pilih satu sambungan dan biarkan bagian lain tenang

Proyek ini sudah terbagi pada sambungan yang berguna. Pengembangan yang baik mengubah bagian terkecil yang bertanggung jawab dan membiarkan tetangganya memakai kontrak yang sudah dikenal.

Sebelum menulis kode, lengkapi kalimat ini: “Perilaku baru berada di _____ karena bagian tersebut sudah memiliki _____.”

Jika jawabannya kredensial Wi-Fi, buka `wifi-manager`. Jika jawabannya alamat HTTP baru, buka `routes`. Jika jawabannya lingkungan build baru, buka implementasi `FirmwareBuilder`.

Bag. 12.2 Sambut chip ESP32 lain

Daftar board saat ini berisi ESP32-S3. Menambahkan target kedua memerlukan lebih dari satu pilihan baru di dropdown.

Tambahkan nama yang diterima ke validasi target. Berikan Rust target dan chip `espflash` yang tepat kepada `TargetSpec`. Tambahkan pilihan board di frontend. Setelah itu, buat pemisahan peripheral runtime yang sesuai dengan chip asli, contoh partisi, dan konfigurasi rollback.

ESP32-C3, misalnya, memakai RISC-V dan tidak membawa kumpulan hardware yang sama dengan S3. Application API hanya boleh menjanjikan sumber daya yang benar-benar dimiliki board.

Mulailah dengan aplikasi kosong yang sudah diketahui bekerja. Buktikan build dan serial flash terlebih dahulu, kemudian uji OTA dan rollback.

Bag. 12.3 Ubah setup Wi-Fi menjadi alur pertama yang ramah

Manager saat ini menerima kredensial pada waktu build dan melakukan reconnect bila diperlukan. Board masa depan dapat bertindak seperti perangkat rumah baru:

1. Cari kredensial tersimpan di NVS.
2. Coba jaringan tersimpan selama waktu yang terbatas.
3. Nyalakan setup access point sementara jika tidak ada jaringan yang berhasil.
4. Tampilkan halaman kecil tempat pemilik memilih Wi-Fi.
5. Simpan kredensial lalu kembali ke station mode.
6. Terbitkan setiap perubahan melalui network handle yang sudah ada.

Biarkan modem, event loop, NVS, dan layanan Wi-Fi berada di dalam `wifi-manager`. Aplikasi sensor tidak perlu ikut mempelajari kode captive portal.

Bag. 12.4 Bawa terminal ESP32 ke dashboard

Ada dua jalan yang berguna. Keduanya memperlihatkan bagian perjalanan yang berbeda.

12.4.1 Jalan pertama: serial bridge pada PC

Sebuah layanan lokal kecil membuka port COM atau tty yang dipilih, membaca baris, lalu mengalirkannya ke browser dengan SSE atau WebSocket. Cara ini dapat menampilkan pesan bootloader, panic, dan kegagalan Wi-Fi sebelum jaringan hidup.

Dashboard harus meminta pengguna memilih port yang akan dibuka. Hanya satu reader boleh memiliki port tersebut. Bridge membutuhkan batas memori, disconnect yang bersih, dan peringatan yang terlihat ketika proses flash tidak dapat dimulai karena monitor masih memegang koneksi serial.

12.4.2 Jalan kedua: log dikirim melalui Wi-Fi

Runtime dapat mengumpulkan catatan log terstruktur, mengirim batch kecil ke Axum, lalu membiarkan browser berlangganan stream milik perangkat. Cara ini bekerja setelah kabel USB dilepas. Kegagalan yang menghentikan Wi-Fi sebelum tersambung tetap tidak dapat dilaporkan melalui jalur ini.

Dashboard yang baik memberi label Serial atau Network pada setiap baris. Server juga membutuhkan batas penyimpanan agar loop yang terlalu berisik tidak memenuhi SQLite atau memori.

Yang tersedia hari ini

Kedua jalur log perangkat tersebut belum diterapkan pada API saat ini. Terminal gelap yang sudah ada hanya berisi build log dari PC. Kalimat ini perlu tetap terlihat agar terminal contoh tidak dikira sebagai bukti dari hardware.

Bag. 12.5 Pindahkan build yang tidak dipercaya ke ruang lebih aman

Saat ini `LocalFirmwareBuilder` menjalankan Cargo pada host PC. Cara tersebut cocok untuk proyek lokal yang dipercaya. Cargo dapat menjalankan build script dan procedural macro, sehingga ZIP dari sumber asing membutuhkan batas yang lebih kuat.

Trait `FirmwareBuilder` menyediakan pintu bagi `DockerFirmwareBuilder`. Container yang dirancang dengan teliti memakai image toolchain yang versinya dikunci, source mount read-only, folder output kecil yang dapat ditulis, serta batas waktu, CPU, memori, process, dan log. Container tidak membawa rahasia milik host.

Coordinator tetap dapat menerima baris log dan artifact akhir dalam bentuk yang sama. Route serta polling frontend tidak perlu mengetahui bahwa ruang build telah diganti.

Bag. 12.6 Berikan tanda tangan sungguhan kepada firmware

Pertahankan SHA-256 untuk integritas transfer. Tambahkan bukti penerbit sebagai lapisan terpisah dengan alat signing yang didukung dan trust anchor pada perangkat.

Sistem build menandatangani image atau digest yang disetujui memakai private key yang dilindungi. Metadata firmware membawa signature, algoritma, dan key ID. Perangkat memverifikasi semuanya sebelum mengaktifkan slot. Rotasi kunci dan pemulihan membutuhkan rencana yang sudah diuji.

Ikuti panduan ESP-IDF Secure Boot dan signing saat proyek bergerak menuju produksi. Rumus shared-secret buatan sendiri terlalu rapuh untuk tugas ini.

Bag. 12.7 Biarkan API tumbuh tanpa mengejutkan proyek lama

Managed upload sudah menyatakan `application_api_version=2`. Ketika kontrak aplikasi berubah, tambahkan atau migrasikan versi dengan sengaja. Berikan pesan penolakan yang jelas kepada ZIP lama beserta contoh konversi pendek.

Utamakan penambahan field yang dapat diabaikan kode lama. Kunci versi runtime dan application API bersama-sama di generated workspace. Jika payload HTTP publik membutuhkan perubahan yang memutus kompatibilitas, berikan batas versi yang mudah terlihat.

Bag. 12.8 Uji dari lapisan murah menuju board

Mulailah dengan unit test untuk path, versi, pembacaan manifest, dan perubahan status. Tambahkan integration test memakai basis data SQLite sementara dan fake builder. Periksa navigasi frontend serta keadaan gagal di browser. Sesudah itu, jalankan proyek yang sudah diketahui baik melalui toolchain ESP asli.

Tes fisik diletakkan terakhir karena memakan waktu paling banyak dan mengungkap jenis masalah yang berbeda. Uji update berhasil, jaringan terlepas, download terputus, hash salah, listrik padam, setup gagal, rollback, dan pertukaran slot berulang.

Tuliskan lapisan mana yang lolos. “Tes Rust berhasil” dan “rollback bekerja pada board ini” adalah dua bukti berbeda.

Coba dahulu sebelum lanjut

Pilih satu pengembangan dari bab ini. Isi kalimat kepemilikan di bagian awal. Di bawahnya, gambar berkas yang akan disentuh, kegagalan baru yang mungkin dilihat pemula, dan tes terkecil yang membuktikan perubahan tanpa hardware. Tambahkan tes fisik pada baris terakhir.



REFERENSI HTTP API

Lampiran ini adalah referensi di samping meja. Anda tidak perlu membacanya dari awal sampai akhir sebelum memakai proyek. Kembalilah ke sini ketika sedang menulis request, memeriksa nama field, atau mencari tahu layanan mana yang memiliki sebuah alamat. Cerita di balik panggilan ini berada pada Bab 3 sampai Bab 9.

Bag. A.1 Konvensi

Dashboard memakai JSON untuk request dan response biasa, multipart form data untuk upload proyek, server-sent events untuk progress CSV, HTML untuk laporan profiling, dan raw octet stream untuk download firmware.

Kegagalan OTA memakai amplop yang sama:

```
{
  "success": false,
  "error": {
    "code": "TARGET_MISMATCH",
    "message": "Device chip does not match firmware target",
    "details": "optional diagnostic detail",
    "hint": "optional corrective action"
  }
}
```

Field tanpa nilai dapat dihilangkan. Semua ID berupa string UUID kecuali `device_id`, yang dipilih melalui konfigurasi atau diturunkan dari station MAC.

Bag. A.2 Indeks endpoint OTA

Metode	Endpoint	Kegunaan
GET	/api/ota/status	Kesehatan layanan dan public URL
GET	/api/ota/projects	Daftar proyek yang sudah di-upload
POST	/api/ota/projects	Upload sebuah ZIP proyek
POST	/api/ota/projects/<project-id>/build	Masukkan build ke antrian
GET	/api/ota/builds	Daftar riwayat build
GET	/api/ota/builds/<build-id>	Status build dan log lengkap
GET	/api/ota/firmware	Daftar firmware siap pakai
GET	/api/ota/firmware/latest?target=esp32s3	Image siap pakai terbaru untuk target
GET	/api/ota/firmware/<firmware-id>	Rincian firmware
GET	/api/ota/firmware/<firmware-id>/download	Stream application image
DELETE	/api/ota/firmware/<firmware-id>	Hapus image yang tidak dirujuk
GET	/api/ota/devices	Daftar perangkat terdaftar
POST	/api/ota/devices/register	Buat atau perbarui perangkat
POST	/api/ota/devices/<device-id>/heartbeat	Segarkan keadaan perangkat
POST	/api/ota/devices/<device-id>/deploy	Pilih firmware yang diinginkan
GET	/api/ota/devices/<device-id>/update	Polling firmware yang diinginkan
POST	/api/ota/devices/<device-id>/ota-status	Laporkan progress atau hasil OTA

Bag. A.3 Status

GET /api/ota/status mengembalikan 200.

```
{
  "service": "CSV_PROFILING OTA Server",
  "language": "rust",
  "status": "online",
  "version": "0.1.0",
  "public_base_url": "http://192.168.1.5:7000",
  "supported_targets": ["esp32s3"]
}
```

Bag. A.4 Project dan build

POST /api/ota/projects mengharapkan field multipart **project**, **target**, dan **version**. Upload yang berhasil mengembalikan 201.

```
curl -F "project=@uploaded-firmware-basic.zip" \
-F "target=esp32s3" -F "version=1.0.1" \
http://localhost:7000/api/ota/projects
```

```
{
```

```
{
  "project_id": "55e4...",
  "name": "uploaded-firmware-basic",
  "target": "esp32s3",
  "version": "1.0.1",
  "status": "uploaded",
  "project_type": "managed_application",
  "application_api_version": 2,
  "entrypoint": "src/main.rs"
}
```

Upload standalone tidak membawa API version dan entrypoint. Nilai `project_type`-nya adalah `standalone`.

GET `/api/ota/projects` mengembalikan array catatan project. Path filesystem internal tidak ikut diserialisasi.

POST `/api/ota/projects/<project-id>/build` mengembalikan 202:

```
{"build_id": "a8c4...", "status": "building"}
```

Baris permanen dimulai dengan status `queued`, lalu coordinator task mengubahnya menjadi `building`. Hanya satu build `queued` atau aktif yang diizinkan bagi satu proyek.

GET `/api/ota/builds` mengembalikan seluruh build, dimulai dari yang terbaru. **GET** `/api/ota/builds/<build-id>` mengembalikan satu respons berbentuk:

```
{
  "build_id": "a8c4...",
  "project_id": "55e4...",
  "project_name": "uploaded-firmware-basic",
  "target": "esp32s3",
  "version": "1.0.1",
  "status": "success",
  "logs": ["Build queued", "Build started", "Tooling: cargo ..."],
  "exit_code": 0,
  "error": null,
  "started_at": "2026-09-07T10:00:00Z",
  "finished_at": "2026-09-07T10:02:00Z",
  "created_at": "2026-09-07T09:59:59Z"
}
```

Respons gagal dapat memuat `hint`.

Bag. A.5 Firmware

Route list, detail, dan latest mengembalikan bentuk firmware yang sama:

```
{
  "firmware_id": "c53b...",
  "project_id": "55e4...",
  "build_id": "a8c4...",
  "project": "uploaded-firmware-basic",
  "target": "esp32s3",
  "version": "1.0.1",
  "filename": "firmware-uploaded-firmware-basic-v1.0.1-a8c4.bin",
  "size": 945332,
}
```

```

    "sha256": "f2534c...",
    "status": "ready",
    "created_at": "2026-09-07T10:02:00Z",
    "download_url": "http://192.168.1.5:7000/api/ota/firmware/c53b.../download"
  }

```

GET /download mengembalikan `application/octet-stream`, content length, attachment disposition, dan X-Firmware-SHA256. Route menyusun kembali path dari build directory yang dikonfigurasi dan filename yang tersimpan, lalu melakukan canonicalization sebelum membuka berkas.

DELETE /api/ota/firmware/<firmware-id> mengembalikan 204. Respons berubah menjadi 409 ketika perangkat atau deployment masih merujuk image tersebut.

Bag. A.6 Perangkat dan deployment

POST /api/ota/devices/register mengembalikan 201 bersama catatan perangkat saat ini.

```

{
  "device_id": "ESP32-S3-001",
  "name": "Sensor Node 1",
  "chip": "esp32s3",
  "current_version": "1.0.0",
  "ip_address": "192.168.1.24"
}

```

POST /api/ota/devices/ESP32-S3-001/heartbeat mengembalikan 200 bersama perangkat yang telah diperbarui.

```

{
  "current_version": "1.0.0",
  "status": "online",
  "ip_address": "192.168.1.24"
}

```

Status heartbeat yang diizinkan adalah online, offline, updating, rebooting, updated, failed, dan unknown.

GET /api/ota/devices mengembalikan catatan dengan field publik berikut:

```

{
  "device_id": "ESP32-S3-001",
  "name": "Sensor Node 1",
  "chip": "esp32s3",
  "ip_address": "192.168.1.24",
  "current_version": "1.0.0",
  "desired_version": "1.0.1",
  "desired_firmware_id": "c53b...",
  "status": "online",
  "ota_status": "pending",
  "ota_progress": 0,
  "ota_error": null,
  "last_seen": "2026-09-07T10:03:00Z"
}

```

POST /api/ota/devices/<device-id>/deploy mengharapkan firmware ID dan mengembalikan 202.

```
{"firmware_id": "c53b..."}
```

```
{
  "deployment_id": "d911...",
  "device_id": "ESP32-S3-001",
  "firmware_id": "c53b...",
  "desired_version": "1.0.1",
  "status": "pending"
}
```

GET `/api/ota/devices/<device-id>/update` mengembalikan `update_available:false` atau manifest yang diperlihatkan pada Bab 9.

POST `/api/ota/devices/<device-id>/ota-status` menerima:

```
{
  "status": "downloading",
  "progress": 35,
  "current_version": null,
  "error": null
}
```

Keadaan OTA yang diizinkan adalah idle, pending, downloading, verifying, installing, rebooting, success, dan failed. Progress harus berada antara 0 dan 100. Status failed mewajibkan error. Status success memakai 100 sebagai nilai bawaan dan mengosongkan desired firmware setelah current version diperbarui.

Bag. A.7 Indeks endpoint CSV

Route berikut dimiliki Flask pada port 5000.

Metode	Endpoint	Kegunaan
GET	/	Sajikan HTML dashboard
POST	/upload	Terima field multipart CSV file dan kembalikan job_id
GET	/progress/<job-id>	Stream progress profiling dengan SSE
GET	/report/<job-id>	Sajikan HTML laporan yang dihasilkan
GET	/sessions	Daftar sesi laporan yang disimpan
GET	/session/<job-id>	Periksa dan pulihkan satu sesi
POST	/clean/<job-id>	Terapkan kontrol cleaning dan perbarui metadata
POST	/playground/<job-id>	Jalankan preview, analisis, chart, atau mutasi yang dipilih jelas
GET	/download-cleaned/<job-id>	Download CSV yang sudah dibersihkan
DELETE	/reset/<job-id>	Hapus satu sesi CSV
DELETE	/reset-all	Hapus seluruh sesi CSV

Field request playground meliputi operation, mutate, limit, column, column_2, value, operator,

`indexes`, dan `chart_type`. Endpoint membatasi nilai limit dari 1 sampai 500.



KONFIGURASI, PERINTAH, DAN GLOSARIUM

Simpan lampiran ini di dekat terminal. Isinya mengumpulkan nama dan perintah yang mudah terlupa setelah gagasan besarnya sudah dipahami. Baca catatan pendek di sekitar perintah sebelum menyimpannya, terutama ketika hardware terhubung.

Bag. B.1 Environment variable

Variable	Nilai bawaan/contoh	Arti
OTA_BIND_ADDRESS	0.0.0.0	Interface listener; akses LAN membutuhkan binding non-loopback.
OTA_PORT	7000	Port HTTP Axum.
OTA_PUBLIC_BASE_URL	terdeteksi atau ditulis	Base URL yang dimasukkan ke alamat firmware bagi perangkat. Gunakan alamat LAN PC.
OTA_PROJECT_DIR	firmware-projects	Root aman bagi upload yang diekstrak.
OTA_BUILD_DIR	firmware-builds	Root aman bagi application image yang dihasilkan.
OTA_DATA_DIR	data	Folder yang memuat ota.db.
OTA_RUNTIME_DIR	examples/esp32-rust-ota-client	Template runtime stabil untuk managed build.
OTA_MAX_UPLOAD_MB	50	Batas field proyek terkode dalam MiB.
OTA_MAX_EXTRACTED_MB	batas upload dikali 10	Jumlah maksimum declared extracted content.
OTA_ALLOWED_ORIGINS	localhost dan 127.0.0.1 pada port 5000	Daftar origin browser yang diizinkan, dipisahkan koma.
OTA_RUST_TOOLCHAIN	esp	Toolchain rustup untuk perintah builder; nilai kosong memakai default process.
WIFI_SSID	kosong	Nama jaringan Wi-Fi embedded yang dipakai saat managed build.
WIFI_PASS	kosong	Kata sandi Wi-Fi embedded. Simpan dalam .env lokal.
DEVICE_NAME	label managed-device	Nama ramah yang ditanam ke managed firmware.

Variable	Nilai bawaan/contoh	Arti
DEVICE_ID	dari MAC bila kosong	Pengganti identitas stabil yang bersifat opsional.
RUST_LOG	info sebagai bawaan	Filter tracing Axum dan tower-http.

Bag. B.2 Lembar perintah

B.2.1 Menyalakan layanan

```
# Windows
.\run-dev.bat

# Linux/macOS
./run-dev.sh
```

B.2.2 Menjalankan pemeriksaan server OTA

```
cargo test --manifest-path ota-server/Cargo.toml
cargo clippy --manifest-path ota-server/Cargo.toml --all-targets
curl http://localhost:7000/api/ota/status
```

B.2.3 Memeriksa alat embedded

```
cargo --version
rustc +esp --version
espflash --version
rustc +esp --print target-list
```

B.2.4 Membangun runtime stabil tanpa flashing

```
cd examples/esp32-rust-ota-client
cargo +esp build --release --target xtensa-esp32s3-espidf
```

B.2.5 Membangun dan melakukan flash runtime stabil

```
cargo +esp run --release --target xtensa-esp32s3-espidf
```

Pengaturan runner menentukan argumen `espflash` dan penggunaan tabel partisi. Periksa `.cargo/config.toml` pada proyek embedded sebelum flashing fisik.

Bag. B.3 Glosarium

Application image Berkas `.bin` yang berisi satu aplikasi ESP-IDF dan cocok untuk partisi aplikasi OTA.

Axum Framework HTTP Rust yang dipakai layanan OTA.

Build ID UUID yang mengenali satu percobaan kompilasi beserta log permanennya.

Cargo target Arsitektur dan platform tujuan kompilasi sebuah crate Rust. Proyek ini dimulai dengan `xtensa-esp32s3-espidf`.

CORS Kebijakan browser yang mengatur origin halaman mana yang boleh memanggil OTA API.

Deployment Hubungan tersimpan yang menyatakan satu perangkat harus memasang satu catatan firmware.

ELF Format executable tertaut yang dipakai oleh `espflashsave-image`.

ESP-IDF Framework pengembangan resmi Espressif dan kumpulan system service yang berada di bawah binding Rust.

Firmware ID UUID yang mengenali satu application image siap pakai beserta metadata-nya.

Flask Framework HTTP Python yang menyajikan dashboard dan CSV API.

Heartbeat Permintaan berkala dari perangkat untuk memperbarui versi, status, alamat IP, dan waktu last-seen.

Inactive slot Partisi aplikasi OTA yang sedang tidak dijalankan dan dapat menerima image baru dengan aman.

Managed application Package Rust berbasis fungsi yang di-upload lalu disatukan server dengan runtime stabil.

NVS Penyimpanan nonvolatile ESP-IDF yang biasa dipakai untuk pengaturan dan konfigurasi Wi-Fi.

OTA Pembaruan firmware melalui koneksi jaringan, singkatan dari over-the-air.

Pull model Perangkat melakukan polling untuk firmware yang diinginkan dan memulai download sendiri.

Rollback Bantuan bootloader untuk kembali ke slot aplikasi valid sebelumnya ketika image baru gagal melewati validasi startup.

SHA-256 Hash yang dipakai untuk memastikan byte hasil download cocok dengan metadata server.

SSE Server-sent events, aliran HTTP satu arah untuk progress profiling CSV.

Stable runtime Kode platform yang pertama kali di-flash dan memiliki Wi-Fi, OTA, rollback, serta siklus hidup aplikasi.

Standalone firmware Proyek Cargo upload yang dibangun tanpa penyatuan managed runtime; proyek memiliki seluruh perilaku perangkatnya.

WAL Mode write-ahead logging SQLite yang membantu akses lokal secara bersamaan.

ZIP slip Path traversal dari arsip yang mencoba mengekstrak berkas keluar dari root tujuan.

Bag. B.4 Batas implementasi saat ini

Versi pertama mendukung ESP32-S3, penyimpanan SQLite lokal, polling build log, HTTP biasa pada LAN yang dipercaya, local trusted build, dan pemeriksaan integritas SHA-256. Streaming terminal perangkat melalui serial atau jaringan, build Docker terisolasi, autentikasi multi-user, publisher signing, dan target chip tambahan adalah pekerjaan pengembangan yang dijelaskan pada Bab 12.

Batas ini adalah garis desain yang terlihat. Pembaca pemula dapat membedakan fitur yang sudah dapat dipakai hari ini dari bagian yang masih membutuhkan rekayasa dan validasi hardware.

CATATAN PENUTUP

Sekarang Anda mempunyai peta dari klik di browser sampai laporan Pandas, serta dari sumber Rust yang diunggah sampai image aplikasi yang ditulis ke slot OTA ESP32 yang sedang tidak aktif. Peta itu lebih berguna daripada hafalan perintah. Saat sesuatu gagal, cari batas tempat kegagalannya terlebih dahulu: browser ke Flask, browser ke Axum, builder ke toolchain, PC ke jaringan lokal, atau bootloader ke aplikasi. Log pada batas tersebut biasanya memberi petunjuk paling jujur tentang langkah berikutnya.

Jaga agar runtime awal selalu dapat dipulihkan melalui USB. Uji rollback pada perangkat nyata. Perlakukan bahan build yang diunggah sebagai kode tepercaya sampai builder benar-benar dijalankan di dalam sandbox. Kebiasaan ini memberi ruang bagi proyek untuk tumbuh sambil menjaga pemisahan yang membuatnya mudah dipahami.